Projekt blackout

Intelligent energistyring ved strømsvigt i kritiske systemer

Softwareteknologi F26

Gruppe: SW10

Dato: 28/5-2026

Version: 1

Antal tegn: 70.795

Gruppemedlemmer

Stud.nr: x	Navn: Anonymt medlem
Stud.nr: x	Navn: Anonymt medlem
Stud.nr: x	Navn: Anonymt medlem
Stud.nr: x	Navn: Anonymt medlem
Stud.nr: x	Navn: Anonymt medlem
Stud.nr: 202410048	Navn: Mark Kruse Schirmer Nielsen
Stud.nr: x	Navn: Anonymt medlem

Vejleder:

Anonym vejleder.

1 Resume

Power Management System (PMS) er et intelligent system, der afgrænser strømtilførslen til rum i kritiske bygninger under strømsvigt for at spare på strømmen. Projektet er en prototype og simulerer virkeligheden i form af tre rum hvor LED'er repræsenterer strømtilførslen eller manglen på samme. Projektrapporten indeholder relevant arkitektur og design for udarbejdelsen af prototypen samt arbejdsprocessen. Gennemgang af rapporten vil give klarhed af baggrunden for udviklingen af prototypen, udviklingsprocessen, design og arkitektur i en grad, hvor enhver fagrelevant person kan gengive udviklingen og nå en funktionsmæssigt tilsvarende prototype.

2 Abstract

Power Management System (PMS) is an intelligent system that limits the flow of power to rooms in critical buildings during power-out with the purpose of saving power. The project is a prototype and simulates reality in the form of three rooms where each room has an LED that represent the flow of power, or lack thereof. The project report contains relevant architecture and design for the completion of the prototype, as well as the work process. Reading the report will give clarity of the cause for the development of the prototype, the development process, design, and architecture in a degree where any qualified person can remake the prototype and reach the same functional level.

Indhold

1 Resume 2
2 Abstract 2
3 Forord 6
4 Projektkoncept 6
4.1 Indledning 6
4.2 Problemformulering 9
4.3 Systembeskrivelse 9
5 Kravspecifikation 11
5.1 Aktørbeskrivelse 11
5.2 Funktionelle krav - Use cases 13
5.2.1 Initier PMS 13
5.2.2 Detekter Aktivitet 14
5.2.3 Regulér strøm 15
5.2.4 Indstil PMS 16
5.2.5 Sluk PMS 17
5.3 Ikke-funktionelle krav 18
5.4 Designskitse af GUI 19
6 Metode og proces 20
6.1 Metode 20
6.2 Proces 20
7 Analyse 22
7.1 Hardware analyse 22
7.1.1 IR-sensor 22
7.1.2 Mikrocontroller 22
7.1.3 Strøm sensor 23
7.2 Software analyse 23
7.2.1 GUI 23
7.2.2 IDE 23
7.2.3 Seriel kommunikation 23

8 Systemarkitektur 24
8.1 Domæne model 24
8.2 Hardwarearkitektur 25
8.2.1 BDD 25
8.2.2 IBD 26
8.3 Softwarearkitektur 27
8.3.1 Klassediagram 27
8.3.2 State Machine Diagram 29
8.3.3 Applikationsmodel for use case 1 “Initiér PMS” 29
8.3.4 Applikationsmodel for use case 2 “Detekter aktivitet” 31
8.3.5 Applikationsmodel for use case 3 “Regulér strøm” 33
8.3.6 Applikationsmodel for use case 4 “Indstil PMS” 34
8.3.7 Applikationsmodel for use case 5 “Sluk PMS” 36
9 Systemdesign 38
9.1 Hardwaredesign 38
9.1.1 Strøm sensor 38
9.1.2 IR sensorer og Multiplexer 40
9.1.3 LED lamper og Relæ 42
9.2 Softwaredesign 47
9.2.1 Power Sensor 47
9.2.2 IR Sensor 52
9.2.3 GUI 55
9.2.4 ProfileData 57
9.2.5 InitPMS 58
9.2.6 PMSControl 61
9.2.7 SetPMS 61
9.2.8 TerminatePMS 63
10 Implementering og modultest 64
11 Integrationstest 64
12 Accepttest 65

12.1 Accepttestspecifikation for funktionelle krav 65
12.2 Accepttestspecifikation for ikke-funktionelle krav 65
13 Resultater 65
13.1 Resultater for accepttest for use cases 65
13.2 Resultater for accepttest for ikke-funktionelle krav 65
14 Diskussion af resultater 66
15 Perspektivering / Fremtidigt arbejde 66
16 Konklusion 68
17 Bilag 69
18 Kildeliste 69

Versionshistorik

Versions nummer	Beskrivelse	Dato
Version 0.1	Udkast til projektbeskrivelse	09-03-2026
Version 1.0	Rettelser efter review 1	11-03-2026
Version 1.1	Udkast til software og hardware arkitektur	07-04-2026
Version 2.0	Rettelser efter review 2	08-04-2026
Version 3.0	Accepttest gennemgået og rettet for at uddybe acceptkrav yderligere	15-05-2026

Tabel 1 - Versionshistorik

3 Forord

Projektet er udarbejdet på Aarhus Universitet, 2. semester af Softwareteknologi. Projektgruppen består af medlemmerne; Anonym medlem, Anonym medlem, Anonym medlem, Mark Kruse Schirmer Nielsen, Anonym medlem, Anonym medlem og Anonym medlem, og er blevet vejledt af Anonym Vejleder. Afleveret den 27-05-2026 og bedømt den 24-06-2026.

Rapporten indeholder kun de væsentligste dele af projektet. For en uddybelse til dele af projektet henvises til bilaget, hvor der kan findes flere detaljer.

4 Projektkoncept

4.1 Indledning

Dette projekt omhandler udviklingen af en prototype af et intelligent strømstyringssystem, der kan håndtere længerevarende strømsvigt i kritiske bygninger. Systemet retter sig særligt mod

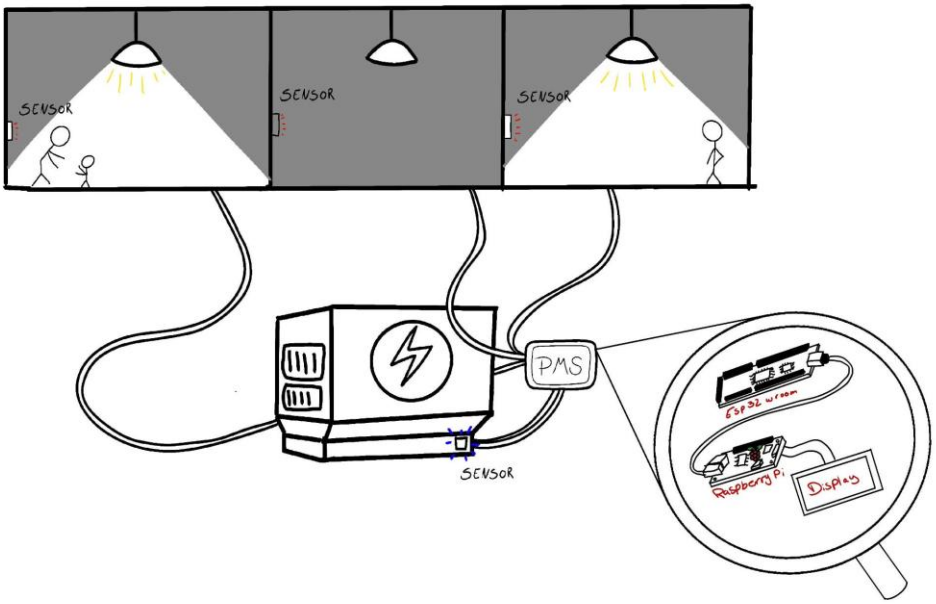
områder, hvor elforsyningen er ustabil, for eksempel i tredjeverdenslande, hvor elnettet ofte er mindre pålideligt end i Danmark. Dog kan strømsvigt og energimangel forekomme overalt, f.eks. ved naturkatastrofer, tekniske fejl eller overbelastning af elnettet. Projektet tager derfor udgangspunkt i en generel problemstilling: *Hvordan sikres det, at begrænset energi, udnyttes optimalt, så essentielle funktioner kan holdes kørende længst muligt?*

Formålet med systemet er at fordele den tilgængelige strøm under strømsvigt, så kritiske rum og funktioner prioriteres frem for mindre essentielle områder. Eksempler på essentielle funktioner kan være medicinsk udstyr, kommunikationssystemer eller serverrum, mens mindre essentielle funktioner kan være venteværelser eller ikke-kritiske kontorområder. Systemet skal kunne afkoble strøm ud fra brugerens valg om hvilke rum, der skal forsynes med strøm, og hvilke der midlertidigt skal frakobles, afhængigt af hvor meget strøm der er tilbage.

Den tekniske løsning bygger på indlejret hardware, som konstant overvåger energiniveauet i en generator eller alternativ energikilde. Ved hjælp af en strømsensor måles det samlede energiforbrug samt den resterende energimængde. Ud fra disse målinger vurderer systemet, hvor meget energi der er tilbage, og regulerer herefter strømfordelingen. Når energiniveauet falder under bestemte grænser, vil systemet automatisk nedprioritere og slukke strømmen til mindre kritiske rum for at bevare strøm til de vigtige funktioner. Derudover anvendes en infrarød sensor til at registrere personer i de enkelte rum. Hvis et rum ikke er i brug, kan systemet automatisk slukke for belysning eller andre ikke-nødvendige installationer for at spare energi.

Systemets logik implementeres i software og fungerer som en central styringsenhed, der analyserer sensordata og kobler strøm til eller fra i realtid. For at sikre fleksibilitet og brugerkontrol udvikles der desuden en grafisk userinterface (GUI), hvor brugeren kan overvåge systemets tilstand, herunder energiniveau, ønskede thresholds og prioriterede rum. Gennem GUI'en kan brugeren manuelt justere thresholds eller slukke for strømmen i specifikke rum, hvis det er

nødvendigt. Thresholds bestemmer, ved hvilket energiniveau systemet nedprioriterer og slukker for mindre kritiske rum.
Projektet kombinerer energistyring, indlejrede systemer og softwarebaseret logik i en samlet løsning med fokus på effektiv energiudnyttelse. Målet er at demonstrere, hvordan intelligent og automatiseret energifordeling kan gøre kritiske bygninger mere robuste under strømsvigt.



Figur 1 - Energistyring under strømsvigt

Designet af projektet er delt ud i delgrupper, og ansvarsområderne kan ses i tabellen nedenfor, men fremgår også i de afsnit hvor bestemte projektmedlemmer har været specielt ansvarlige.

GUI	Anonymt medlem
	Anonymt medlem
	Anonymt medlem
IR sensor	Anonymt medlem
	Anonymt medlem
	Anonymt medlem
Strøm sensor	Mark Kruse Schirmer Nielsen
	Anonymt medlem

Tabel 2 - Fordeling af ansvarsområder

4.2 Problemformulering

I dette projekt laves optimering af energiforbrug ved længerevarende strømsvigt i kritiske systemer. Formålet er at optimere fordelingen af tilgængelig energi, så essentielle rum og funktioner bliver prioriteret under strømsvigt. Systemet anvender en kombination af strøm- og infrarød sensor til at skabe den rette prioritering af strøm til diverse afdelinger i en relevant bygning, som også kan styres manuelt gennem en brugergrænseflade. De infrarøde sensorer bruges til at detektere mennesker i et givent rum så systemet kan sende strøm til diverse rum. Strømsensor anvendes yderligere til at detektere mængden af strømforbrug og mængden af strøm tilgængeligt, dermed sker overvågning via sensorerne.

4.3 Systembeskrivelse

Systemet er designet til intelligent energistyring i kritiske situationer, hvor den primære strømforsyning svigter. Power Management Systemet (PMS) fungerer som den centrale kontrolenhed, der prioriterer strømfordelingen fra en Emergency Power til specifikke rum baseret på realtidsdata og brugerdefinerede profiler.

PMS:

ESP32-WROOM-32 varetager den eksekverende logik og dataindsamling. ESP32 modtager strøm fra Raspberry Pi'en og er direkte forbundet til alle sensorer. Det er denne enhed, der modtager og analyserer signaler fra de fysiske sensorer og på baggrund heraf styrer strømmen til de enkelte rum.

Raspberry Pi 4b fungerer som systemets overordnede kontrol- og kommunikationsenhed. Den håndterer den grafiske userinterface (GUI) og behandler brugerinput. Raspberry Pi'en er master for displayet og sender relevante styringssignaler videre til systemets logiske enhed ESP32. Strømmen til selve PMS forsynes gennem Raspberry Pi'en, som herefter distribuerer strøm til de øvrige styringskomponenter.

TouchDisplayet er monteret direkte på Raspberry Pi og udgør den primære interaktionsflade. Gennem displayet er det muligt at overvåge strømforbrug og vælge mellem fem redigerbare strømstyringsprofiler, der definerer thresholds af de enkelte rum.

ACS712 Hall Effect-sensoren måler det aktuelle strømforbrug i systemet. Hvilket bruges til at lave et kvalificeret bud på hvor stor restkapacitet, der er tilbage i Emergency Power.

Multiplexer PCA9548A fungerer som en central I^2C-switch, der gør det muligt for ESP32 at kommunikere med de tre MLX90640-sensorer individuelt. Da alle kameraerne har den samme statiske I^2C-adresse, ville en direkte parallelkobling medføre adressekonflikter. Multiplexeren løser dette ved at opdele bussen i otte uafhængige kanaler, hvor systemet softwaremæssigt

kan skifte mellem SD0, SD1 og SD2 for at indsamle termiske data fra hvert rum sekventielt uden datatab eller interferens.

MLX90640 Thermal Camera sensorerne (32x24 IR array) anvendes til præcis tilstedeværelsesdetektering i de opkoblede rum. I modsætning til almindelige bevægelsessensorer måler disse sensorer varmestråling i 768 pixels, hvilket gør det muligt for systemet at detektere personer, selv når de sidder stille. Sensorerne leverer temperaturdata til ESP32, hvilket muliggør intelligent styring af strømmen til et rum baseret på faktisk tilstedeværelse og den valgte profil.

Eksterne dele:

LED-indikatorerne anvendes som visuel demonstration af strømfordelingen. Hver LED-indikator repræsenterer strømforsyningen til et rum, hvilket gør det muligt at observere PMS-enhedens beslutninger i praksis, når den åbner eller lukker for strømkredsløb.

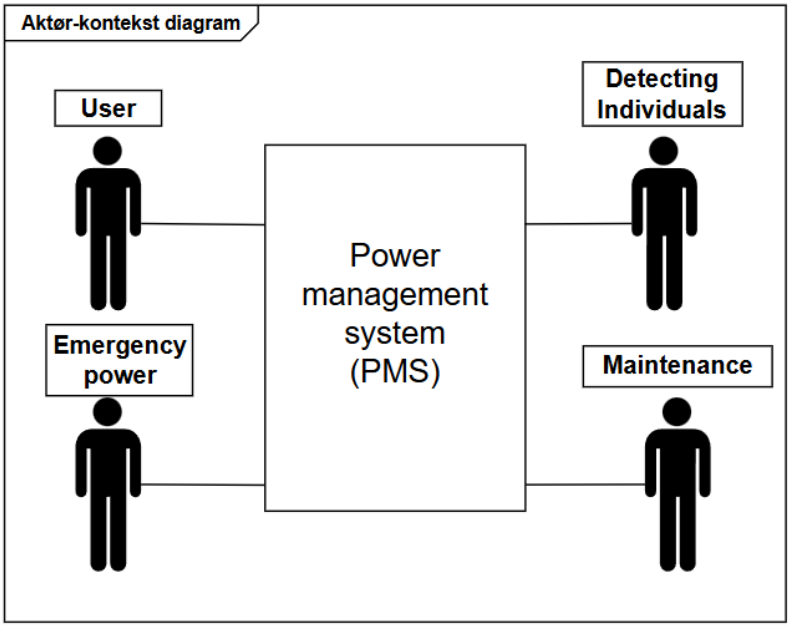
Relæ systemet fungerer som broen mellem den digitale styring (ESP32) og selve effektdistributionen fra nødstrømsløsningen.

Nødstrømsløsning simuleres i testopsætningen ved hjælp af stabil strømtilførsel fra en spændingsforsyning. Denne strømkilde leverer udelukkende strøm til LED-indikatorerne og er elektrisk adskilt fra strømmen til selve PMS-styringen. Dette sikrer, at kontrolsystemet forbliver operativt, uafhængigt af belastningen på nødstrømskilden. Prototypen er lavet med udgangspunkt i et batteri som nødstrømsløsning.

5 Kravspecifikation

5.1 Aktørbeskrivelse

Figur 2 viser et aktør-kontekstdiagram for PMS samt en tabel med aktørbeskrivelser.



Figur 2 - Aktør-kontekst diagram

Aktør navn:	User
Type:	Primær aktør
Beskrivelse:	User er den primære bruger, og det er User, som interagerer med systemet mest. User i dette tilfælde vil være den relevante organisation som styrer den kriseramte bygning, fx IT-afdelingen på et hospital. User kan indstille thresholds for, hvornår forskellige prioriteringsprofiler initieres.

Tabel 3 - Aktørbeskrivelse "User"

Aktør navn:	Emergency Power
Type:	Primær aktør
Beskrivelse:	Emergency Power er en primær aktør, da den interagerer med systemet på samme niveau som User. Når et blackout sker, initieres systemet af nødstrømmen. Emergency Power er en primær bruger, da den er direkte ansvarlig for aktiveringen af PMS.

Tabel 4 - Aktørbeskrivelse "Emergency Power"

Aktør navn:	Detecting individuals
Type:	Sekundær aktør
Beskrivelse:	Detecting individuals er sekundære brugere. Detecting individuals rolle er kun at facilitere systemets prioritering af strøm på baggrund af om der er mennesker i et rum eller ej.

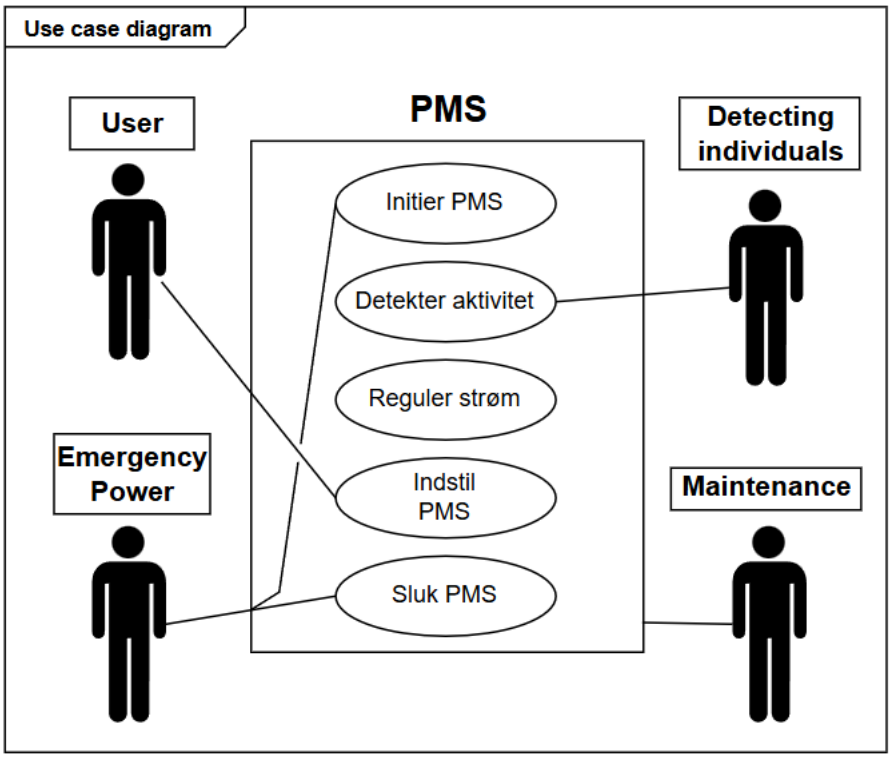
Tabel 5 - Aktørbeskrivelse "Detecting individuals"

Aktør navn:	Maintenance
Type:	Sekundær aktør
Beskrivelse:	Maintenance er aktøren, som kan tilkaldes, hvis use case 1 fejler og systemtjekket rapporterer, at der er fejl i systemet. Maintenance er defineret som en sekundær aktør, da Maintenance kun interagerer med systemet i en udvidelse af use case 1.

Tabel 6 - Aktørbeskrivelse "Maintenance"

5.2 Funktionelle krav - Use cases

På figur 3 kan ses et use case diagram for alle use cases i PMS samt en efterfølgende fully-dressed use case for hver use case.



Figur 3 – Use case diagram

5.2.1 Initier PMS

Navn	Initier PMS
Mål	At initiere PMS
Initiering	Emergency power
Aktører	Emergency power (Primær), Maintenance (Sekundær).
Antal samtidige forekomster	1 (Systemet initieres af emergency power ved et blackout).
Prækondition	Alle profiler i PMS er præindstillet til at alle rum slukker ved 0% og profil 1 er aktiv.

	PMS fungerer normalt og strømsensoren er tændt.
Postkondition	PMS er aktiveret og overvåger aktivt rummene.
Hovedscenarie	1. Et blackout rammer bygningen og Emergency Power aktiveres, samtidigt aktiveres PMS. 2. Et systemcheck startes. 3. Systemet udfører unittests af alle sensorer og et systemcheck af softwaren. 4. Efter fuldført check, informeres User via displayet, og User kan nu anvende systemet. [Undtagelse 1: Fejl i system] 5. De forudindstillede thresholds og prioriteter aktiveres og overvåges af PMS.
Udvidelser/undtagelser	1. Fejl i system a. Systemet rapporterer fejl i unit tests, hvilket informeres om via pop-up.
Datavariationsliste	• Strømoutput fra Emergency Power (ampere/volt) • Threshold (tal)

Tabel 7 - Fully dressed use case "Initiér PMS"

5.2.2 Detekter Aktivitet

Navn	Detekter Aktivitet
Mål	At detektere om der er mennesker i et rum.
Initiering	Use Case 1 - Initier PMS
Aktører	Detecting individuals
Antal samtidige	3 (idet der i alt er 3 sensorer, der skal detektere, om der er aktivitet i 3 forskellige rum)

forekomster	
Prækondition	PMS er initieret.
Postkondition	Use case ”Regulér strøm”
Hovedscenarie	1. Sensorerne måler, om der er aktivitet eller ej i et rum. 2. Sensorerne skal sende signal til PMS om, at der er/ikke er detekteret aktivitet.
Udvidelser/undtagelser	Ingen.
Datavariationsliste	• Internt signal fra sensorerne til PMS.

Tabel 8 - Fully dressed use case "Detekter aktivitet"

5.2.3 Regulér strøm

Navn	Regulér strøm
Mål	At strømmen bliver tilsluttet ønskede rum.
Initiering	Initieres af use Case 2 - Detekter Aktivitet
Aktører	User (primær), Detecting individuals (sekundær)
Antal samtidige forekomster	3 (ud fra antallet af sensorer)
Prækondition	Use case 2 “Detekter aktivitet” er udført.
Postkondition	At der er / ikke er strøm i de(t) ønskede rum.
Hovedscenarie	1. Signal modtages fra sensor i et rum. 2. Strøm tændes eller slukkes i rum.
Udvidelser/undtagelser	Ingen

Datavariationsliste	• Signal fra sensoren (boolean)

Tabel 9 - Fully dressed use case "Regulér strøm"

5.2.4 Indstil PMS

Navn	Indstil PMS
Mål	At indstille PMS prioriteringer og thresholds.
Initiering	User
Aktører	User (rolle: IT-admin)
Antal samtidige forekomster	1 (Da GUI styres fra en Raspberry Pi, kan kun én User anvende GUI'en koblet på PMS ad gangen.)
Prækondition	At PMS kører normalt og er klar til at blive aktiveret.
Postkondition	PMS er indstillet til den ønskede profil.
Hovedscenarie	1. User vælger "Settings" på GUI'en for PMS. 2. User vælger en af de 5 profiler, og indstiller thresholds for hvert rum. [Undtagelse 1] 3. User vælger "Save", indtaster password og profilen gemmes på PMS. [Undtagelse 2] 4. User kan til hver en tid gå ind under "Settings" og genindstille thresholds for en given profil. 5. Den ønskede aktive profil vælges på Home page fra "profile" dropdown menuen
Udvidelser/undtagelser	A. Forkert threshold input a. User indtaster en threshold-værdi der ikke er et heltal imellem 0 og 100 B. Forkert pinkode a. User taster forkert pinkode og data gemmes ikke

Datavariationsliste	• Profilnavne (tegn) • Thresholds (tal)

Tabel 10 - Fully dressed use case "Indstil PMS"

5.2.5 Sluk PMS

Navn	Sluk PMS
Mål	At slukke PMS
Initiering	Emergency power (slukkes)
Aktører	Emergency power (slukkes)
Antal samtidige forekomster	1 (Systemet slukkes af Emergency Power ved et blackout)
Prækondition	PMS har været tændt
Postkondition	PMS er slukket.
Hovedscenarie	1. Strømmen i bygningen returneres og Emergency Power slukker, hvilket PMS detekterer. 2. PMS gør klar til at lukke alle systemer forsvarligt og returnerer til "idle"
Udvidelser/undtagelser	Ingen.
Datavariationsliste	

Tabel 11 - Fully dressed use case "Sluk PMS"

5.3 Ikke-funktionelle krav

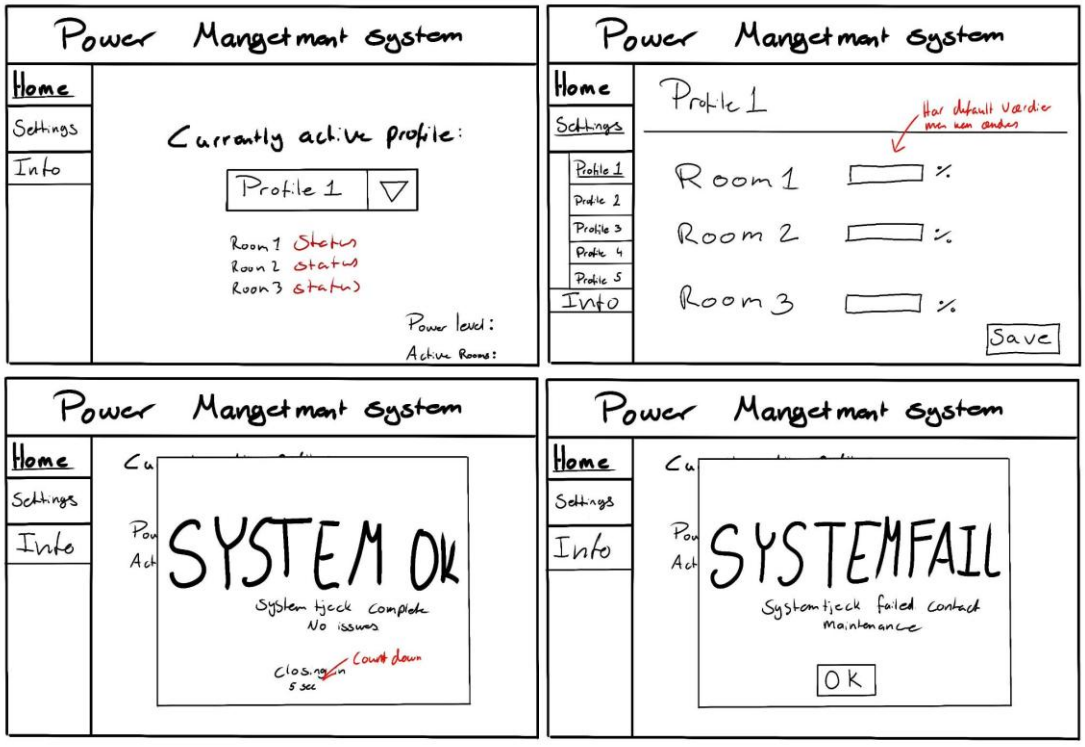
Nr.	Ikke-funktionelle krav	MOSCOW	URPS+
U1	Ved fuldført godkendt systemtjek vises beskeden "System booted succesfully. Closing window automatically in 5 seconds." på GUI'en på Raspberry Pi displayet og beskeden lukkes automatisk efter 5 sekunder.	MUST	Usability
U2	Ved fuldført ikke godkendt systemtjek vises beskeden "System boot failed. Contact system maintenance about hardware issues." på GUI'en på Raspberry Pi displayet.	MUST	Usability
U3	GUI'en skal have minimum én undermenu.	MUST	Usability
U4	GUI'en skal have en statistik undermenu.	COULD	Usability
U5	User skal indtaste en adgangskode for at kunne gemme de ændrede thresholds.	MUST	Usability
U6	GUI'en skal indeholde 5 redigerbare strømstyrings profiler.	MUST	Usability
U7	GUI'en skal være let at navigere i og forstå.	SHOULD	Usability
R1	Hvert rum, hvor PMS skal være aktivt, skal være udstyret med en infrarød sensor, der kan detektere individer i en afstand af 30 cm +/- 5 cm.	MUST	Reliability
R2	PMS skal lave et systemtjek ved opstart for at verificere systemets funktionalitet, og at alle komponenter fungerer korrekt.	MUST	Reliability
R3	Når en person kommer ind foran en IR-sensor, sendes signal til PMS, som tænder for strøm i rummet indenfor max 5 sekunder.	MUST	Reliability
R4	Når den sidste person forlader rummet, skal IR-sensoren sende signal til PMS, som slukker strømmen til rummet efter en brugerdefineret mængde tid, indenfor intervallet 5 – 600 sekunder.	SHOULD	Reliability

R5	Strømsensor skal kunne detektere hvor meget strøm, der er tilbage i batteriet med +/- 3%.	COULD	Reliability
P1	Når User laver en ændring på GUI'en, bør ændringen træde i kraft efter maks. 5 sekunder.	SHOULD	Performance
P2	PMS bør aktiveres inden for 5 sekunder af Emergency Power start	SHOULD	Performance
P3	Systemtjek bør være gennemført maksimalt 70 sekunder efter systemstart.	SHOULD	Performance
P4	PMS bør deaktivere efter maksimalt 5 sekunder	SHOULD	Performance
S1	Der skal være fysisk adgang til sensorerne i tilfælde af nødvendig vedligeholdelse.	MUST	Supportability

Tabel 12 - Ikke funktionelle krav

5.4 Designskitse af GUI

Den originale skitse af brugergrænsefladen, se figur 4, blev anvendt som udgangspunkt for udvikling af systemets GUI. Den endelige prototype ligner skitsen, men der er blevet foretaget ændringer under udviklingsprocessen for at forbedre funktionaliteten.



Figur 4 - Skitse af GUI

6 Metode og proces

6.1 Metode

Til udarbejdelsen af projektet har SysML og UML været essentielle værktøjer, der har gjort det muligt at give alle en lige forståelse for projektets indhold. Ved brug af SysML og UML er der blevet udarbejdet; kravspecifikation, domænemodel, BDD, IBD, state machine diagram, sekvensdiagram og applikationsmodel. Disse diagrammer, udarbejdet i fællesskab i projektgruppen, har givet alle en lige forståelse og mulighed for at udvikle produktet. Ligeledes ville disse diagrammer gøre det muligt for enhver fagperson at udvikle produktet, uanset om vedkommende har deltaget i udarbejdelsen af disse diagrammer. Dette er hvad, der gør SysML og UML til kraftige værktøjer, og det er af denne årsag, vi har valgt at anvende SysML og UML til projektet.

Dog er der blev afveget lidt fra UML-standarden i applikationsmodellen, idet GUI'en er blevet udarbejdet i Python, uden brug af objektorienteret programmering af hensyn til "best practise". Hele GUI'en er illustreret som klassen UserInterface i applikationsmodellen. UserInterface er da ikke en reel klasse, men en repræsentation for GUI'en samt de metodekald, der er essentielle for forståelsen for eksekveringen af PMS.

Derudover kaldes kontrolklasserne, som ses i sekvensdiagrammerne ikke ved samme navn, som den use case de tilhører, men derimod kaldes kontrolklasserne ved et tilsvarende engelsk navn. Dette er blevet gjort sådan, at alle klasser og alt kode er på engelsk. Dog er betydningen stadig den samme.

6.2 Proces

Gruppen blev dannet på forhånd af semesterkoordinatoren, baseret på vores insights-profiler, hvilket var med til at skabe en gruppe med forskellige styrker og svagheder. I starten af projektet udarbejdede vi en samarbejdskontrakt, hvilket bidrog til, at vi som gruppe gik ind til projektet med samme tilgang og mindset. Kontrakten gav os et godt grundlag for, hvordan vi ønskede at fungere som gruppe og arbejde hen imod projektets mål.

I starten af projektet arbejdede vi som gruppe sammen på problemformulering, kravspecifikation og arkitektur. Dette sikrede, at alle gruppemedlemmer havde en fuld forståelse af projektet og det mål, vi gerne ville opnå. Da vi nåede til designfasen, inddelte vi gruppen i tre undergrupper: strøm-sensor, IR-sensor og brugeroverflade. Denne opdeling gjorde det muligt at optimere arbejdsprocessen, og nå i mål med alle delopgaver.

Projektets tidsplan blev udarbejdet som et Gantt-diagram, hvilket var med til at skabe overblik over projektforløbet. Her indsatte vi alle overordnede arbejdsopgaver og markerede, hvilke

perioder de enkelte opgaver skulle udføres, og hvornår de skulle færdiggøres. Derudover anvendte vi arbejdsmetoden kanban til de specifikke arbejdsopgaver. Vi anvendte hjemmesiden 'Trello', hvor vi tilføjede kolonnerne "TODO", "TODO-rapport", "Startet" og "Done". Her indsatte vi de forskellige arbejdsopgaver og kunne med tiden rykke dem mellem kolonnerne og markere dem som færdige i forhold til, hvor langt vi var i arbejdsprocessen.

I gruppen afholdt vi et ugentligt møde med vores vejleder. Tiden blev brugt til at stille spørgsmål, diskutere hvordan forskellige problemstillinger skulle håndteres og få sparring til valg og beslutninger. Efter vi havde opdelt os i undergrupper, blev de ugentlige møder også brugt til at følge op på hinandens fremgang, og informere hinanden om mulige ændringer eller opdateringer i undergrupperne. Dette fungerede godt, da alle gruppemedlemmer dermed fik indsigt i projektets samlede fremdrift og kunne sikre, at alle arbejdede i samme retning.

Vi har arbejdet med en kollektiv projektledelse i gruppen, da vi ikke havde en fast projektleder gennem forløbet. Denne arbejdsform fungerede godt i vores gruppe, da alle medlemmer deltog aktivt gennem forløbet. Til at administrere projektet anvendte vi en fælles OneDrive mappe, hvor alle filer og dokumenter blev delt, dermed havde alle medlemmer adgang til alt. Til kode arbejdet anvendte vi gitlab.au.dk, da vi nemt og overskueligt kunne arbejde parallelt på koden. Intern kommunikation i gruppen foregik på Facebook Messenger, hvilket gjorde kommunikation mellem gruppemedlemmerne hurtigt og effektivt. Kommunikation med vores vejleder foregik på mail, når vi havde spørgsmål eller til planlægning af møder.

7 Analyse

7.1 Hardware analyse

7.1.1 IR-sensor

For en dybdegående begrundelse for valget af IR-sensor henvises til Bilag 1.A – Analyse. Den endelige konklusion på processen blev, at IR-sensoren MLX90640 Thermal Cameras var den bedst egnede sensor, da målingerne var meget præcise og levede op til kravet om at kunne detektere, hvorvidt en person var i et rum, med brug af infrarød stråling, i en tilfredsstillende grad. En lille ændring i bevægelse blev detekteret, også på længere afstand end de andre testede sensorer. Da disse termiske kameraer levede op til kravet i projektet, blev disse det endelige valg af IR-sensorer.

7.1.2 Mikrocontroller

Valget af mikrocontroller stod mellem Arduino Mega 2560 og ESP32-WROOM-32.

I systemet skulle mikrocontrolleren kommunikere med både Raspberry Pi 4 og den termiske sensor MLX90640 Thermal Camera. Her var logikniveau en vigtig faktor, da både Raspberry Pi og MLX90640 anvender 3.3V logik. Arduino Mega 2560 anvender 5V logik, hvilket ville kræve en spændingstilpasning mellem komponenterne for at undgå at beskadige hardwaren. ESP32 anvender 3.3V logik og kunne derfor integreres direkte med Raspberry Pi og MLX90640.

Derudover er hardwarespecifikationerne analyseret med fokus på databehandling og kommunikationshastighed. Ud fra mikrocontrollernes datablade kan denne tabel opstilles;

Specifikation	Arduino Mega 2560[1]	ESP32-WROOM[2]
Clock hastighed	16MHz	240MHz
RAM	8 KB SRAM	520 KB SRAM
Operativ spænding	5V	3.3V
Baud rate	115200	921600

Tabel 13 - Hardware specifikationer for Arduino Mega 2560 og ESP32-WROOM

I tabel 13 kan der aflæses at, ESP32 har en markant højere clock hastighed og mere RAM end Arduino Mega 2560. Dette gør ESP32 bedre egnet til at behandle større datamængder fra MLX90640-sensorerne. Samtidig giver den højere baud rate mulighed for hurtigere serielkommunikation mellem ESP32 og Raspberry Pi'en.

[1]Bilag 1.F Datablade i " ArduinoMega2560Datasheet"
[2]Bilag 1.F Datablade i " ESP32-wroom-32_datasheet_en"

På baggrund af analysen blev ESP32-WROOM valgt som mikrocontroller, da den opfylder flere af systemets krav.

7.1.3 Strøm sensor

Strøm sensorens opgave er at give et bud på den resterende kapacitet i Emergency power. Dermed er strøm sensorens vigtigste opgave at oversætte strømmen, der bliver trukket i systemet til noget mikrocontrolleren, kan forstå. Derfor er ACS712 smart, da den oversætter strøm til en spænding, som kan bruge en ADC-port på ESP32 til at få data ud af.

7.2 Software analyse

7.2.1 GUI

Til udvikling af GUI stod valget mellem Python bibliotek Tkinter, som arbejder godt sammen med Raspberry Pi eller HTML, som er god til at udvikle hjemmesider. Idet der arbejdes med Raspberry Pi med display, blev Python biblioteket Tkinter anvendt.

Python og Tkinter blev valgt på baggrund af de mange online ressourcer om samarbejdet mellem Tkinter og Raspberry Pi hardware. Fordelen med Python lå også i at der på forhånd var kendskab til Python for alle medlemmer af gruppen, hvor at ikke alle fra gruppen havde samme kendskab til HTML.

7.2.2 IDE

I forbindelse med skift af mikrocontroller valgtes Arduino IDE v2.3.9 i stedet for den tidligere bekendte MicroChip Studio IDE, på baggrund af nyere og nemmere design. Arduino IDE'en benyttes til udvikling af koden for hovedlogikken lokaliseret på ESP32. Denne IDE fungerer som en "wrapper" omkring C++, hvorfor der i Arduino IDE 2.3.9. stadig bruges programmeringssproget C++ på trods af den anderledes opsætning af koden. For uddybende begrundelse af valg af IDE henvises til Bilag 1.A - Analyse afsnit 1.2.1 "*Valg af IDE*".

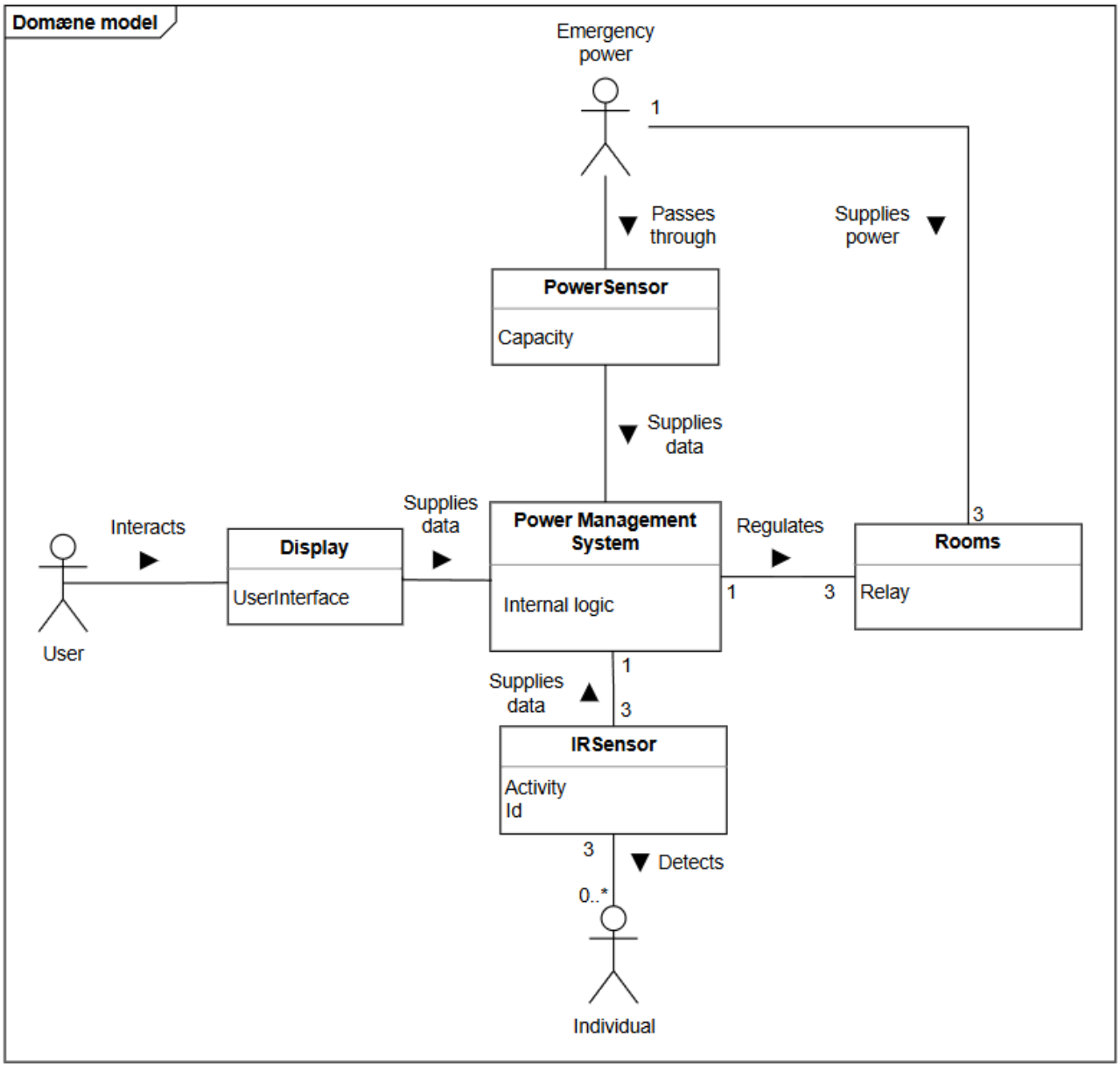
7.2.3 Seriel kommunikation

Raspberry Pi og ESP32-WROOM kommunikerede igennem en USB-forbindelse frem for via Bluetooth. Hovedårsagen til dette er for at skabe et samlet system, hvor ESP32 får strøm igennem Raspberry Pi frem for at have sin egen separate strømkilde. Bluetooth forbindelse kan også til tider være ustabil, f.eks. kan enhederne afbryde forbindelsen ved dårlig forbindelse, hvorimod de ved USB kun mister forbindelsen, hvis man manuelt afkobler kablet eller kablet bliver beskadiget. Da PMS er afhængig af stabil kommunikation imellem ESP32 og Raspberry Pi, var USB det sikre valg, specielt da PMS skal agere ved strømsvigt i kritiske bygninger, er det vigtigt, at forbindelsen er stabil og konstant.

8 Systemarkitektur

8.1 Domæne model

På figur 5 ses en overordnet domænemodel for PMS hvor både hardware og software indgår for at give et helhedsoverblik over systemet sådan, at diagrammet kan give et indblik i hele systemet på ét diagram.

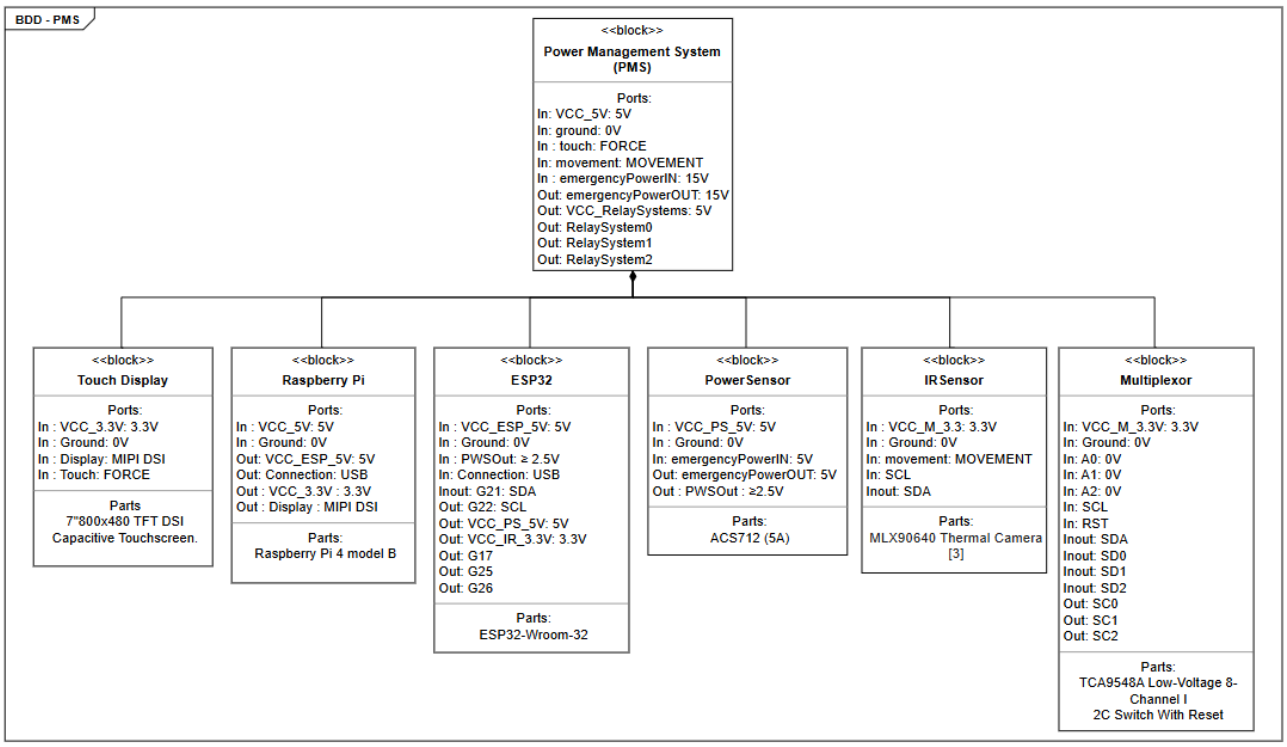


Figur 5 - Domæne model

8.2 Hardwarearkitektur

8.2.1 BDD

På figur 6 ses et BDD over PMS hvor alle blokke og deres relation til hinanden kan ses. I hver blok ses desuden, hvilke forbindelser den pågældende blok indeholder.



Figur 6 - BDD

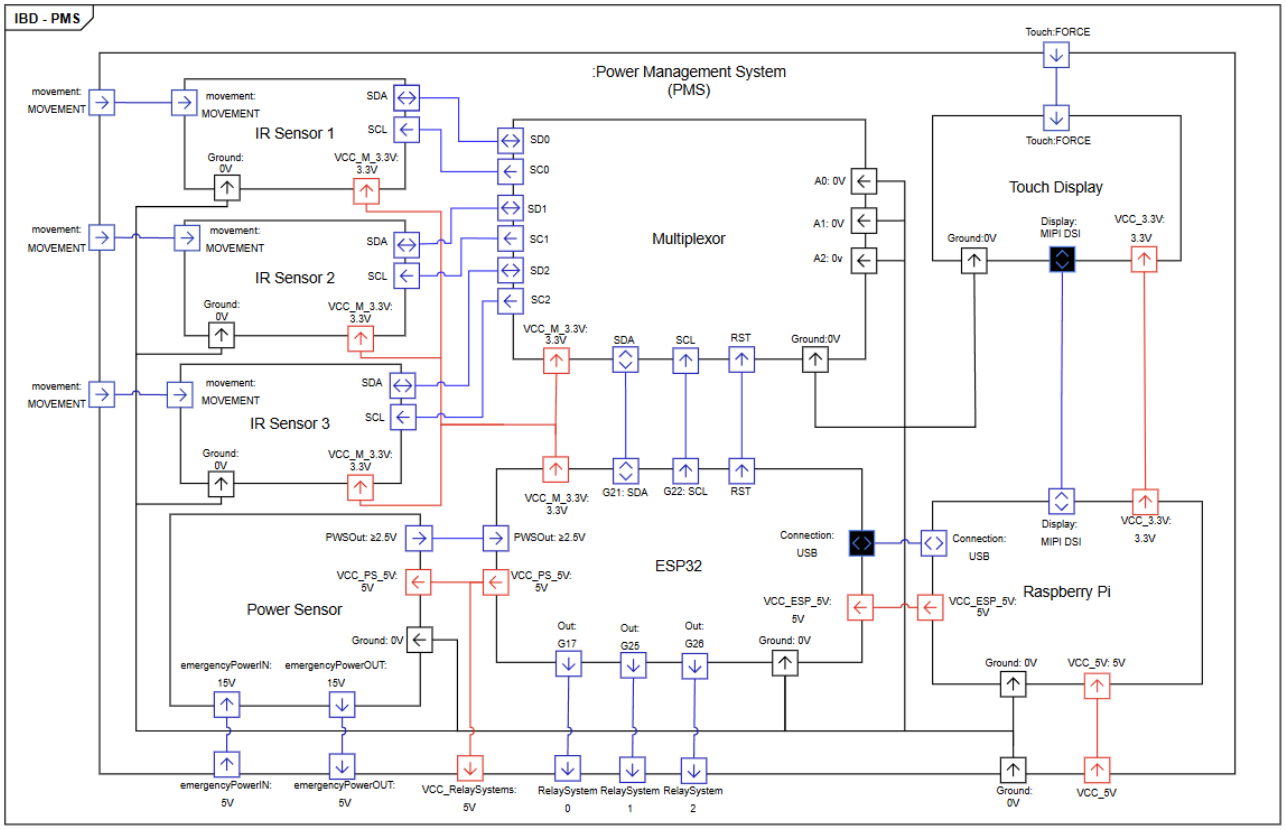
Blokbeskrivelse af BDD

Blok-navn	Funktionsbeskrivelse
Power Management System	Selve systemet.
ESP32	Mikrocontroller enhed som modtager og behandler signaler fra sensorer og GUI.
Raspberry Pi	Enhed som faciliterer GUI'en som User anvender.
Touch Display	Enhed som viser GUI til User.
PowerSensor	Sensor som overvåger hvor meget strøm der er tilbage i batteriet.
IRSensor	Infrarød sensor som tjekker om der er individer i et givent rum.
Multiplexor	Skaber I^2C forbindelse mellem de 3 infrarøde sensorer og ESP32.

Tabel 14 - Blokbeskrivelse af BDD

8.2.2 IBD

IBD, som ses på figur 7, illustrerer en separat USB, strøm og groundforbindelse mellem ESP32 og Raspberry Pi, selvom ESP32 får strøm, ground og kommunikerer via ét samlet USB-stik. Dette er illustreret separat, så der ikke er nogen tvivl om, hvor ESP32 får strøm fra.



Figur 7 – IBD

Signalbeskrivelse af IBD

Signalbeskrivelse af de vigtigste signaler kan ses i tabel 15 - den fulde signalbeskrivelse kan findes i Bilag 1.B - Arkitektur.

PMS	Signal-navn	Funktion	Område	Port 1 (Source)	Port 2 (Destination)	Kom-men-tar
	In movement: MOVEMENT	Detekterer individer gennem IR-sensorer		Detecting in-dividuals	IR Sensor	
	Connection: USB	Kommunikation imel-lem Raspberry Pi og ESP32.		Raspberry Pi	ESP32	
	Inout: SD0	I²C signal fra multiple-xeren	3.3V	ESP32 pin G21	Multiplexor SDA	

	Out: SC0	Bestemmer serial clock (frekvens til kommunikation) mellem multiplexer channel 0 og IR sensor		Multiplexer pin SC0	IR Sensor pin SCL	
	PWSOut: ≥2.5V	Output fra Power Sensor som angiver resterende strøm kapacitet af batteriet.	≥2.5V	Power Sensor	ESP32	
	Display: MIPI DSI	Input fra Raspberry Pi til visuelt display på Touch Display	8-18 bit	Raspberry Pi	Touch Display	

Tabel 15 - Signalbeskrivelse af IBD

8.3 Softwarearkitektur

8.3.1 Klassediagram

InitPMS indeholder et array med tre IRSensor objekter. Disse klasser har et kompositionsforhold, da "child"-klassen InitPMS er en underklasse af "parent"-klassen IRSensor.

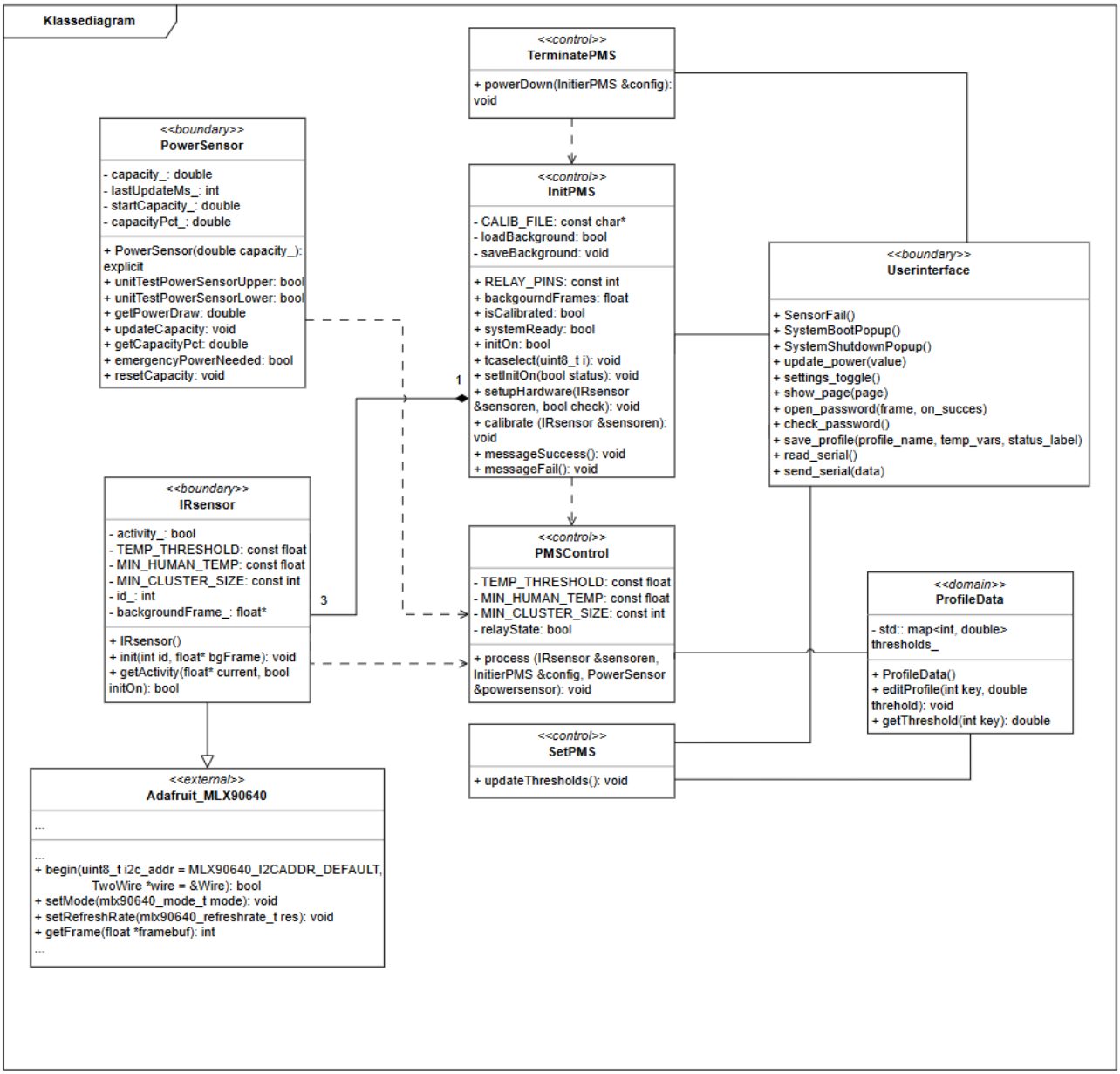
PMSControl afhænger af IRSensor, InitPMS og Powersensor til at udføre nødvendige funktioner, dermed dependency forholdet mellem PMSControl og de 3 andre klasser.

TerminatePMS afhænger af InitPMS, idet systemet først kan slukke (TerminatePMS), hvis det først har været tændt (InitPMS).

IRsensor klassen er en nedarvning af den offentlige klasse Adafruit_MLX90640, idet IRsensor klassen skal bruge nogle af funktionerne fra den offentlige klasse i forbindelse med databehandling. Da anvendes metoderne fra Adafruit ikke i kommunikationen imellem de andre klasser, inkluderes Adafruit kun i det endelige klassediagram, men ikke i applikationsmodellen.

De resterende klasser UserInterface, ProfileData og SetPMS har et associationsforhold til TerminatePMS, InitPMS og PMSControl da de blot kommunikerer med hinanden.

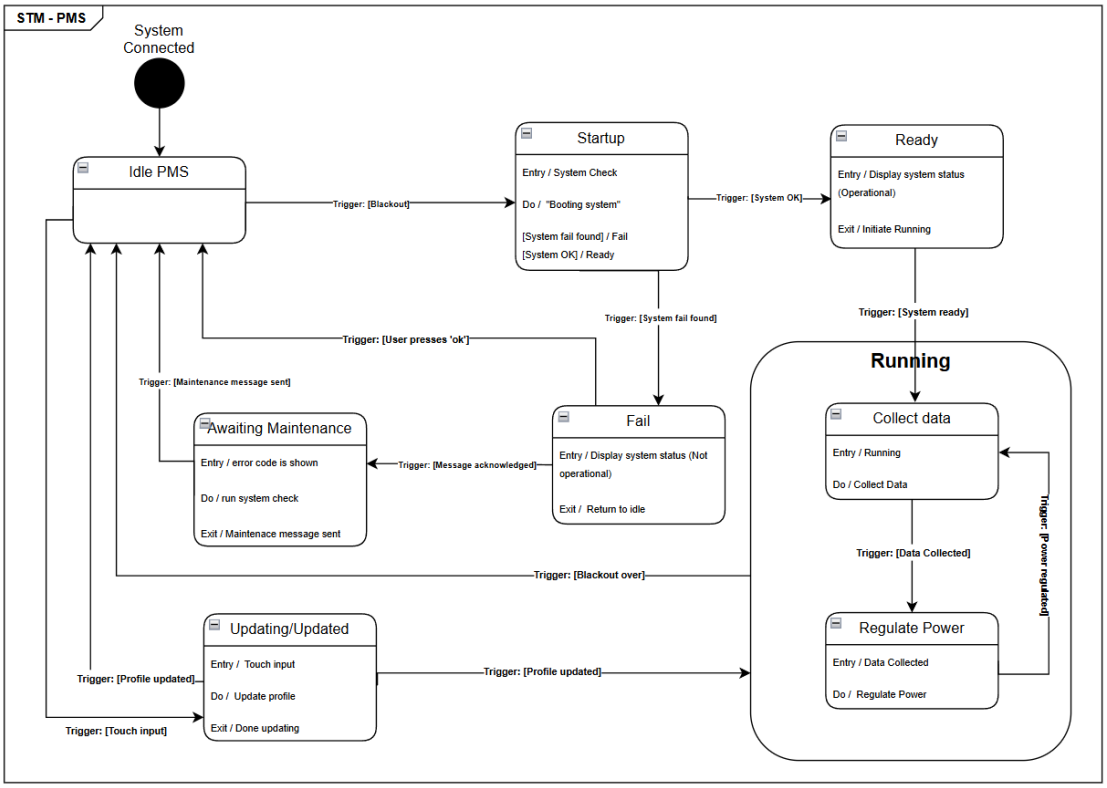
Som nævnt i metode afsnit 6.1 er UserInterface ikke en reel klasse, men en repræsentation for GUI'en med dens vigtigste metoder.



Figur 8 - Klassediagram for PMS.

8.3.2 State Machine Diagram

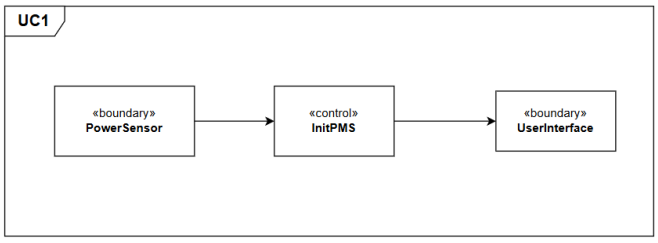
Figur 9 viser et state machine diagram for PMS. Diagrammer illustrer systemets forskellige tilstande ”states”, samt de triggere der forsager overgange mellem de tilstande. Systemet starter i ”Idle”, gennemfører ”Startup” og systemcheck. Ved ”System OK” går systemet til ”running” tilstand, hvor der samles data og strøm reguleres i et loop inde i “running”. Diagrammet viser også ”Updates”, samt håndtering af fejl ”Awaiting maintenance”.



Figur 9 - State Machine Diagram for PMS.

8.3.3 Applikationsmodel for use case 1 “Initiér PMS”

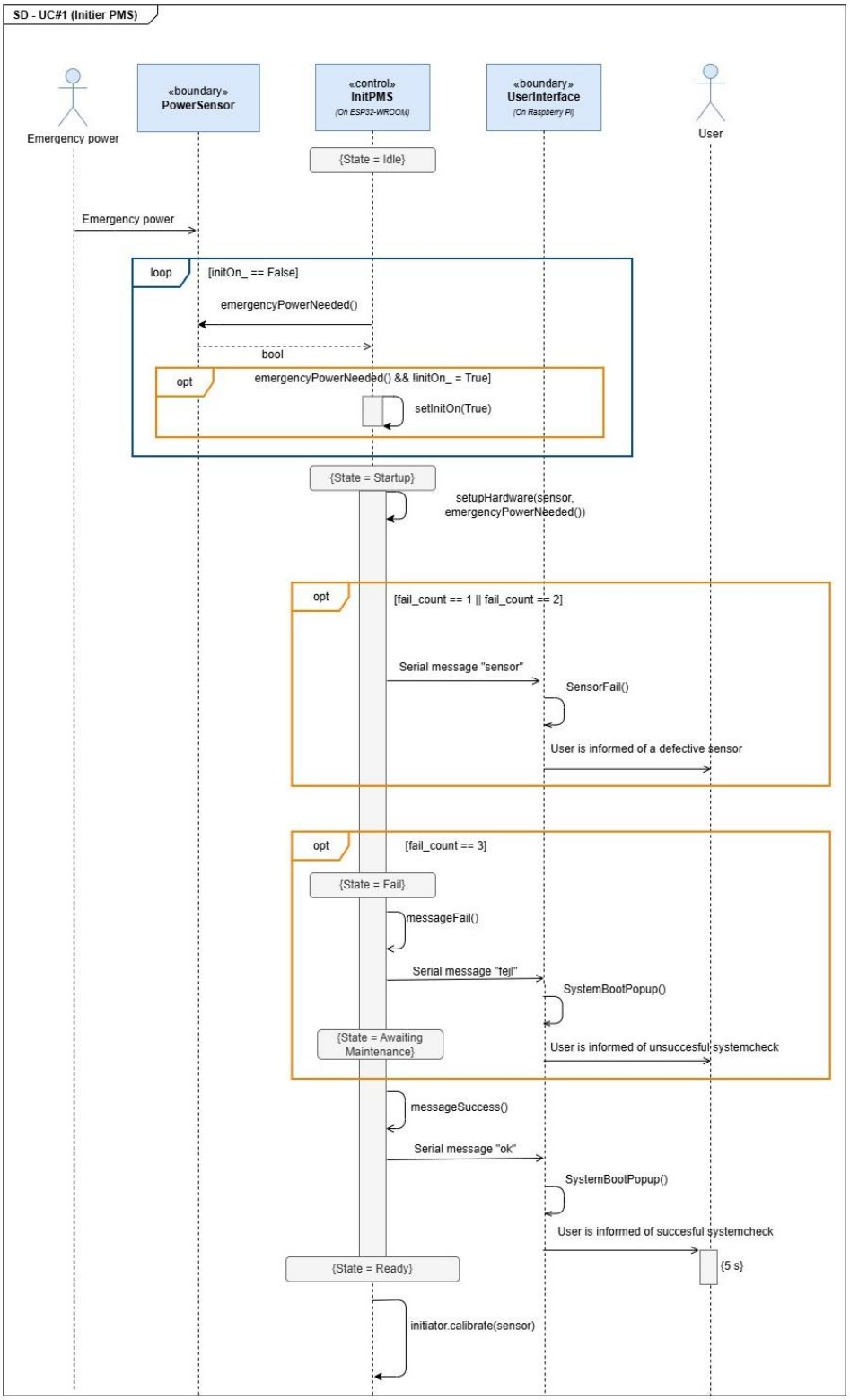
På figur 10 ses de klasser der er aktive under use case 1. Her ses det, at InitPMS henter data fra PowerSensor klassen og sender data ud til den konceptuelle UserInterface.



Figur 10 - Klassediagram for use case 1

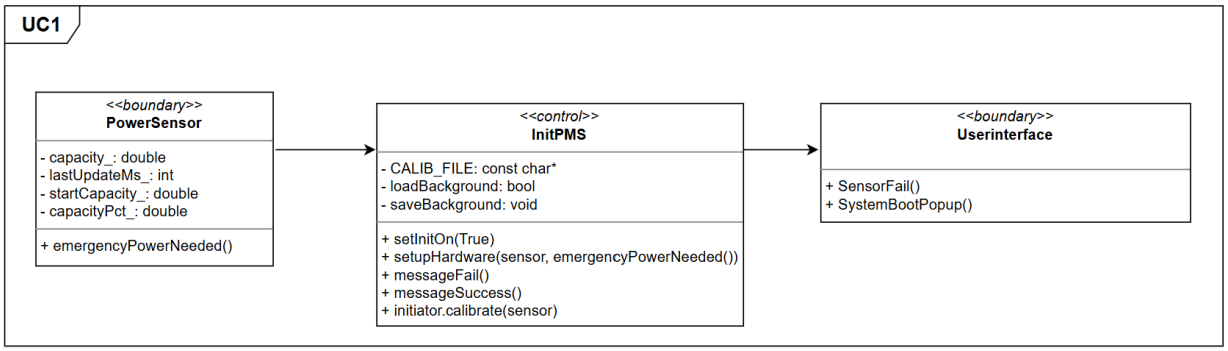
På figur 11 ses sekvensdiagram for use case 1 ”Initiér PMS”. Sekvensdiagrammet giver overblik over, hvordan de relevante blocks kommunikerer i denne use case. Der ses også en række

af de undtagelser, som use casen kan løbe ind i. Herunder sensor-fejl, strøm-sensor fejl eller komplet systemfejl, som vil udløse state Awaiting Maintenance.



Figur 11 - Sekvensdiagram for use Case 1

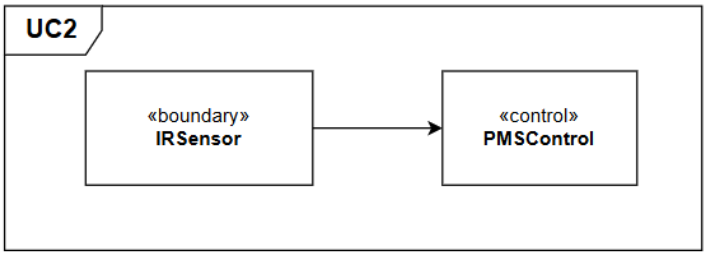
På figur 12 ses et klassediagram for use case 1. Klassediagrammet indeholder nu de metoder og attributter som er blevet anvendt i sekvensdiagrammet.



Figur 12 – Udvidet klassediagram for use case 1

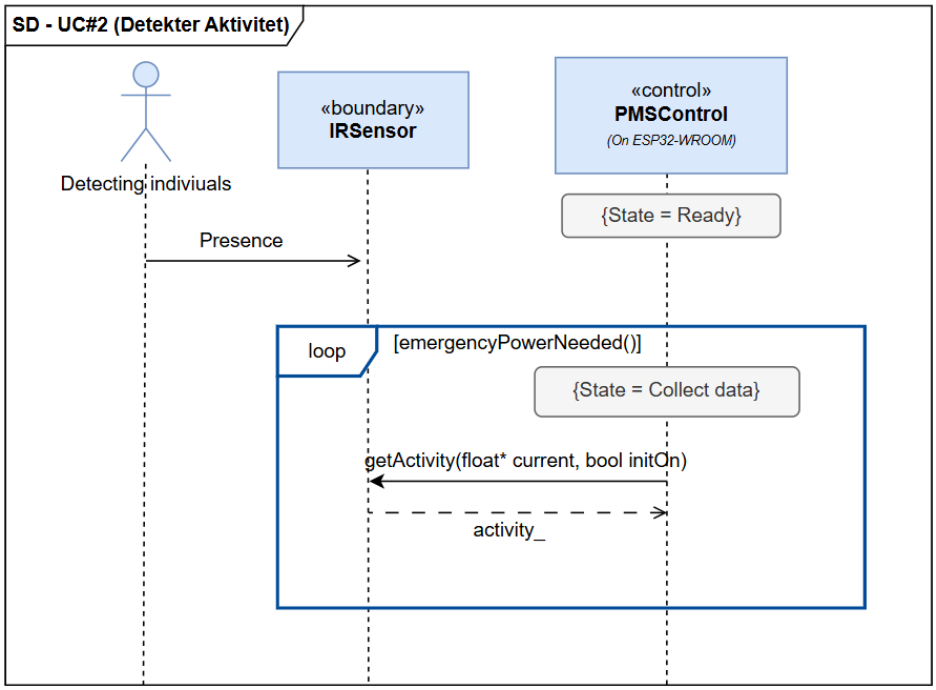
8.3.4 Applikationsmodel for use case 2 "Detekter aktivitet"

På figur 13 ses en klasse oversigt over de anvendte klasser i use case 2. Her ses det at PMSControl henter data fra boundary klassen IRSensor.



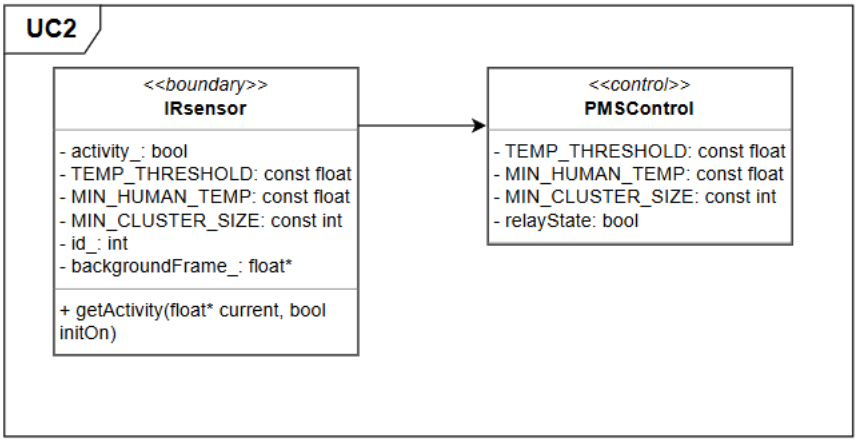
Figur 13 - Klassediagram for use case 2

På figur 14 ses sekvensdiagrammet for use case 2. Her ses det at der samles data fra IRSensor så længe, der er behov for Emergency Power. IRSensor opfanger, at der er en person i rummet og giver denne information videre til PMSControl.



Figur 14 - Sekvensdiagram for use Case 2

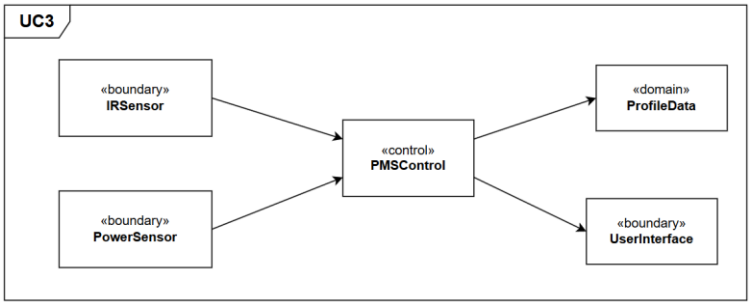
På figur 15 ses et udvidet klassediagram, som nu indeholder attributter og metoder fra sekvensdiagrammet.



Figur 15 - Udvidet klassediagram for use case 2

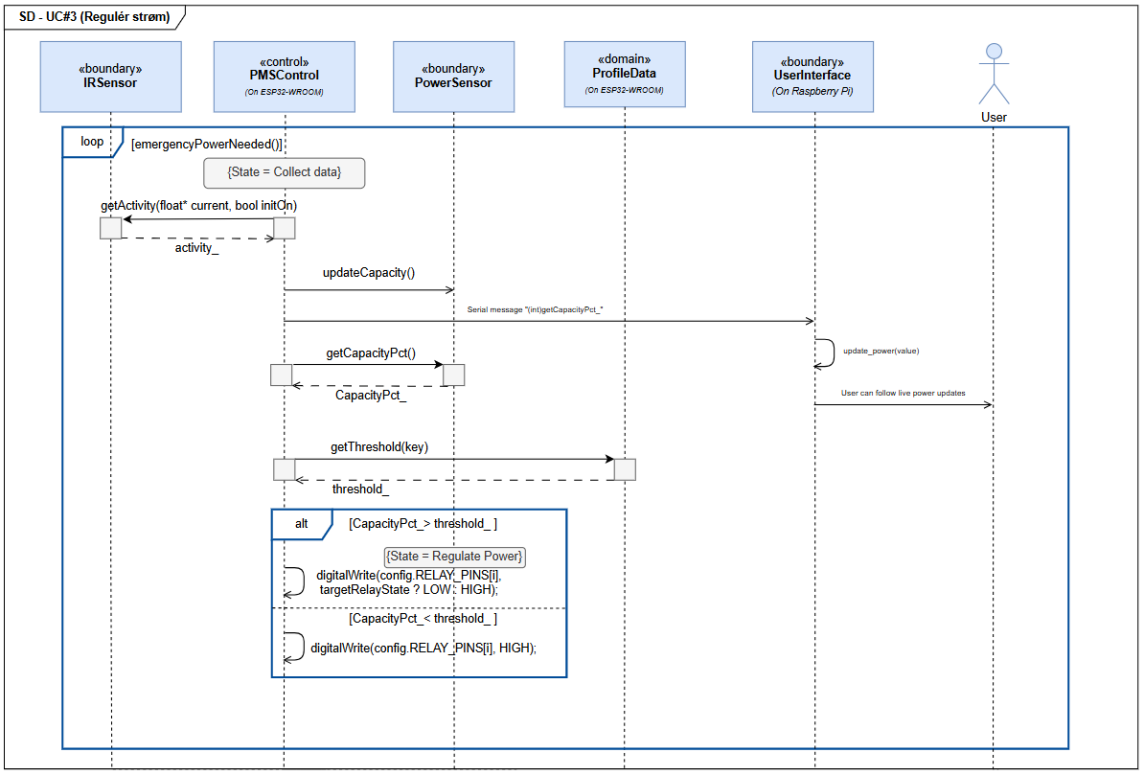
8.3.5 Applikationsmodel for use case 3 “Regulér strøm”

På figur 16 ses klasserne fundet på baggrund af use case 3 “Regulér strøm”. Begge typer af sensorer fungerer som en boundary, da de interagerer med aktører. PMSControl modtager da data fra sensorerne og gemmer dataen i ProfileData eller sender ordrer til UserInterface, som også interagerer med aktører.



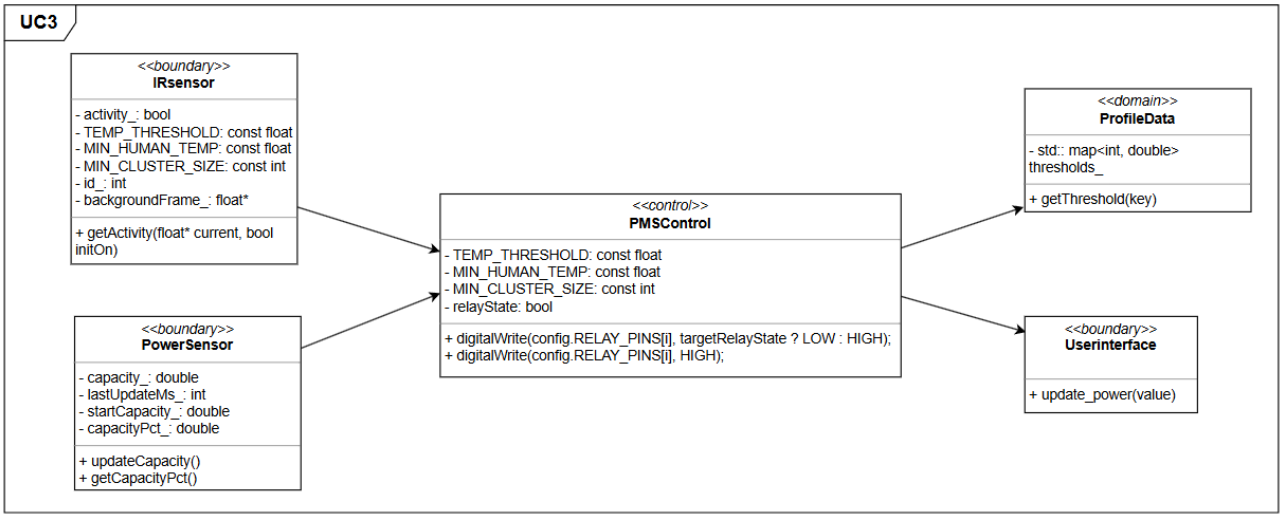
Figur 16 - Klassediagram for use case 3

På figur 17 vises sekvensdiagrammet for eksekveringen af use case 3. Regulér strøm kører konstant i baggrunden, hvor der tjekkes strømniveauet, om Emergency Power er tilkoblet, samt om der er aktivitet i de pågældende rum. PMSControl henter diverse data fra PowerSensor, herunder opdateres den hypotetiske rest kapacitet af Emergency power, som lagres i PowerSensor, samt reflekteres på UserInterface, hvor User konstant kan se strømniveauet. Derudover hentes en nyligt opdateret *capacityPct*, som senere bruges til at tjekke om et threshold hentet fra ProfileData er oversteget.



Figur 17 - Sekvensdiagram for use Case 3

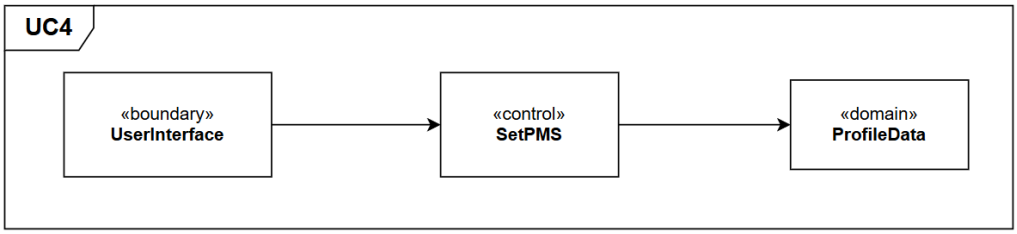
På figur 18 ses det opdaterede klassediagram, som nu indeholder attributter og metoder fra sekvensdiagrammet.



Figur 18 - Udvidet klassediagram for use case 3

8.3.6 Applikationsmodel for use case 4 "Indstil PMS"

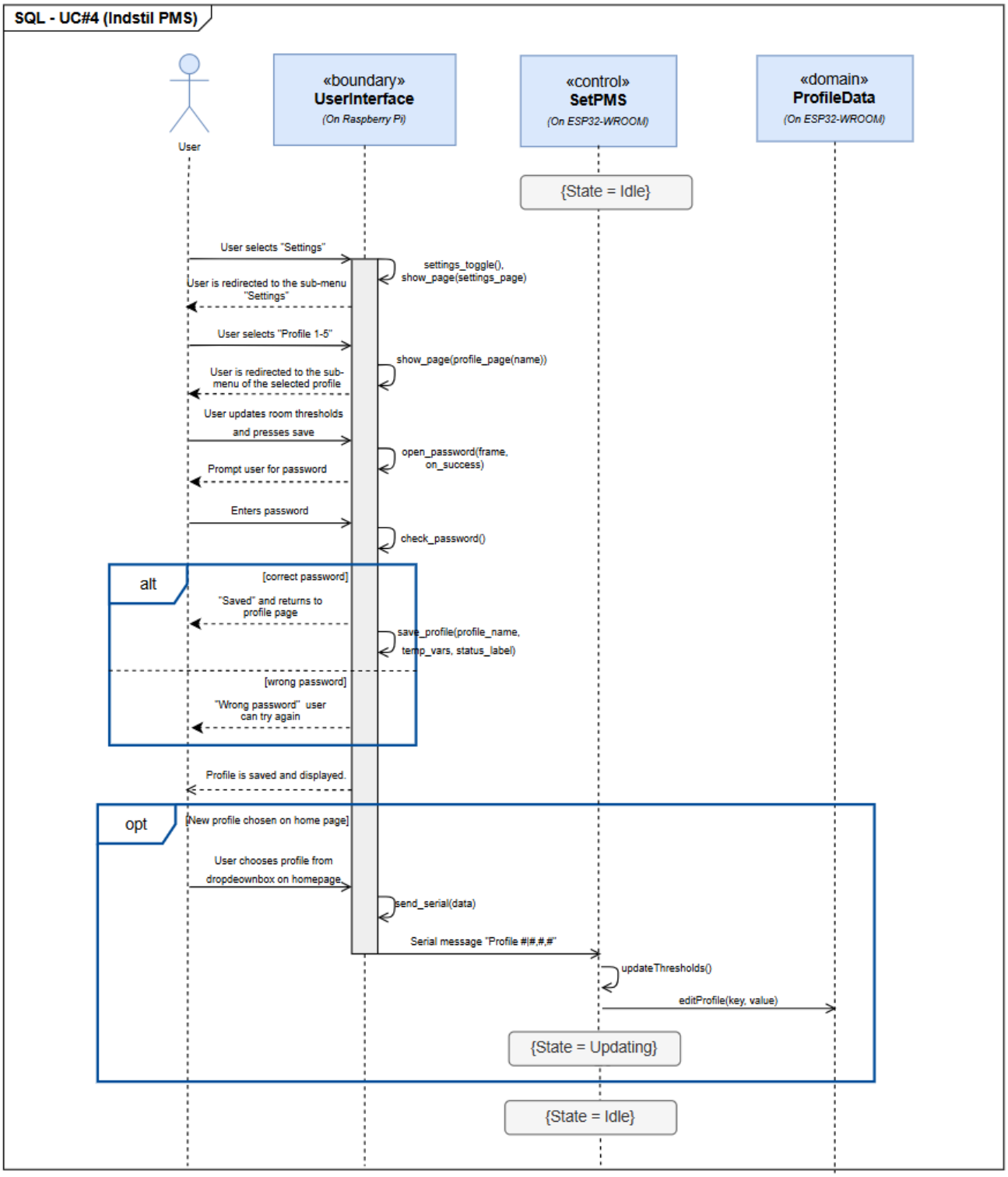
På figur 19 ses klasserne fundet ud fra use case 4 "Indstil PMS". UserInterface er grænsen imellem User og systemet og er derfor en boundary. SetPMS kontrollerer dataet fra UserInterface og gemmer det i domænet ProfileData som holder data.



Figur 19 - Klassediagram for use case 4

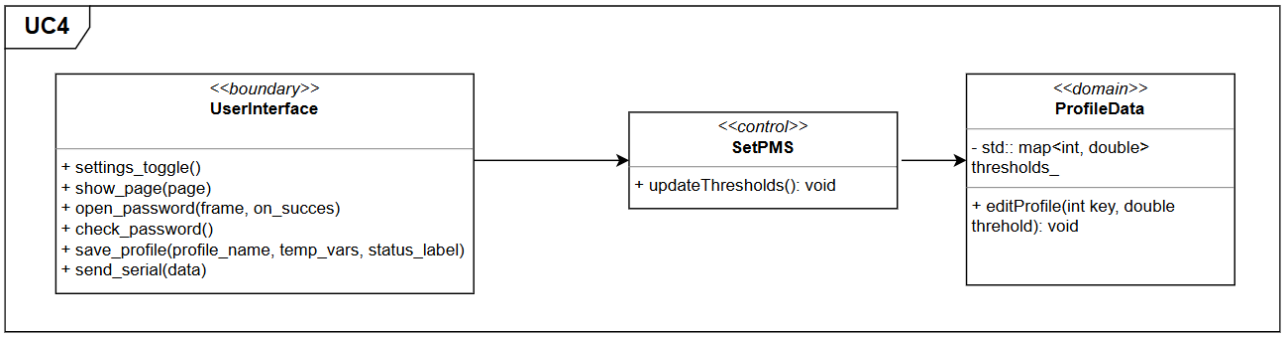
På figur 20 vises sekvensdiagrammet med forløbet af use case 4, som starter med at User vælger en profil og ændrer den. Givet User indtaster den korrekte pinkode og derefter vælger

en ny aktiv profil, bliver dataet fra den aktive profil sendt videre til kontrolklassen SetPMS som til gengæld gemmer dataet i domænet ProfileData.



Figur 20 - Sekvensdiagram for use Case 4

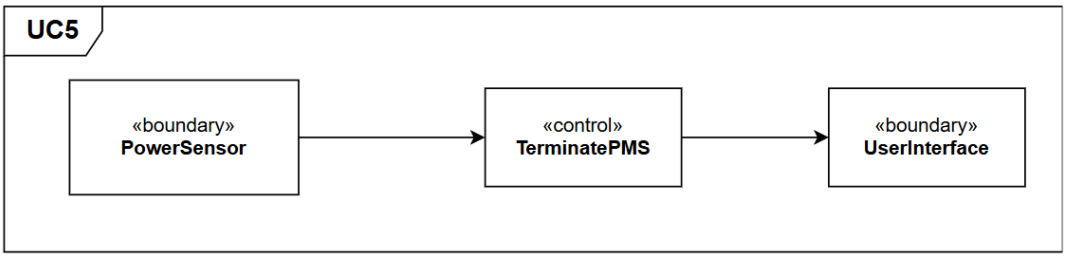
Figur 21 viser det opdaterede klassediagram med metoderne fra sekvensdiagrammet.



Figur 21 - Udvidet klassediagram for use case 4

8.3.7 Applikationsmodel for use case 5 "Sluk PMS"

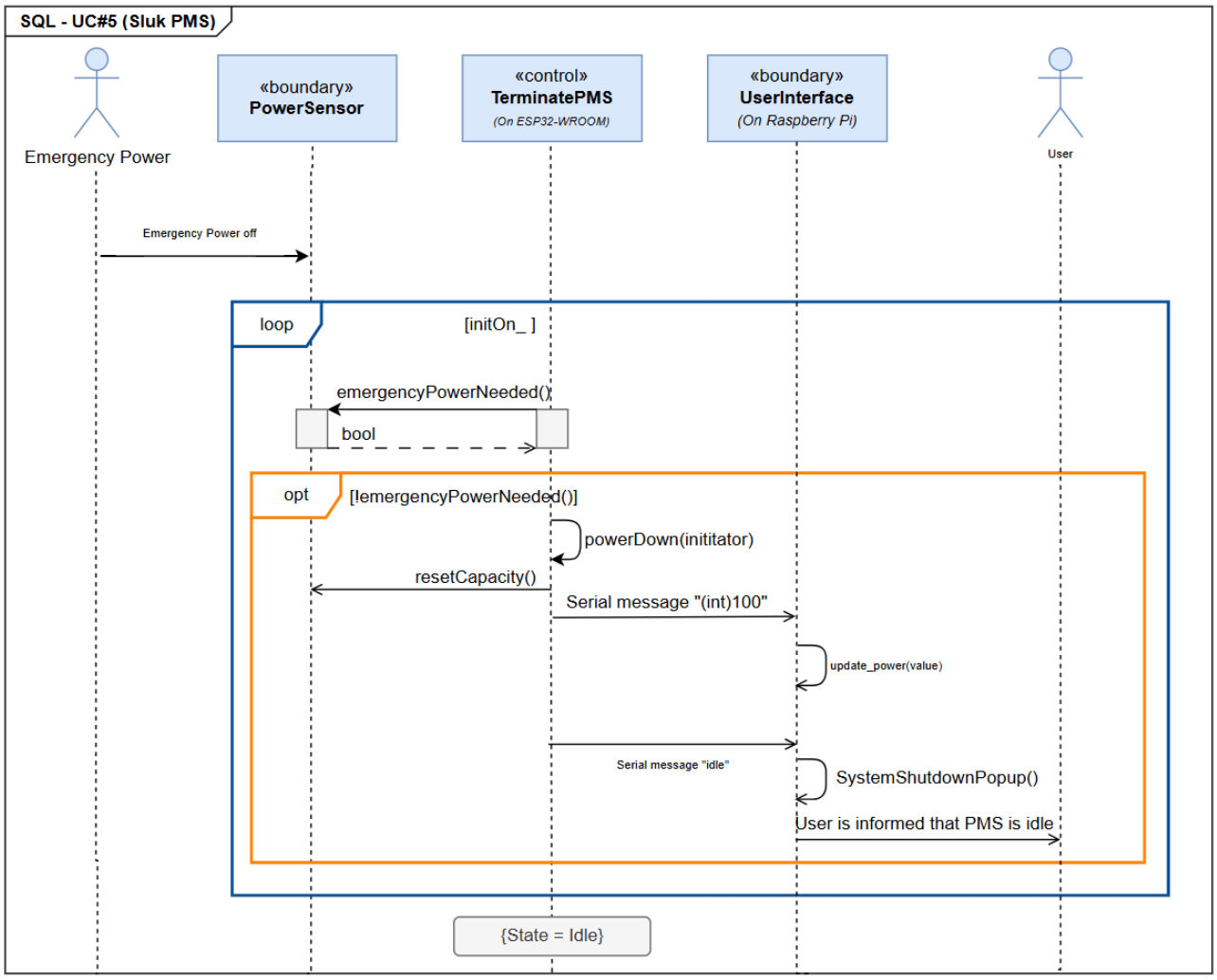
På figur 22 nedenfor ses de klasser der er fundet ud fra use case 5 "Sluk PMS".



Figur 22 - Klassediagram for use case 5

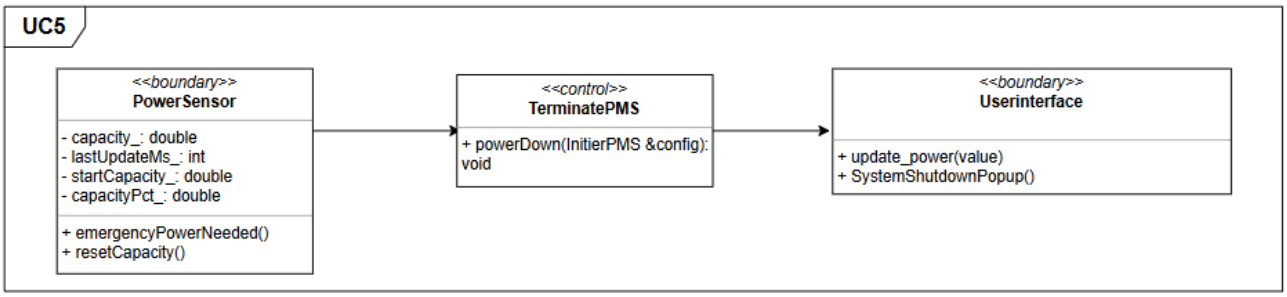
PowerSensor leverer data til TerminatePMS, når TerminatePMS anmoder om det, og sender derefter relevant data videre til den konceptuelle klasse UserInterface, som er grænsen imellem User og PMS.
Sekvensdiagrammet nedenfor viser, hvordan Emergency Powers interagerer med PowerSensor, hvorefter TerminatePMS tjekker om Emergency Power stadig er aktivt. Hvis Emergency

Power er slukket, igangsætter TerminatePMS afbrydelsen af systemet og informerer UserInterface, hvorefter UserInterface informerer User.



Figur 23 - Sekvensdiagram for use Case 5

Figur 24 viser de opdaterede klasser med de nye metoder fundet i sekvensdiagrammet.



Figur 24 - Udvidet klassediagram for use case 5

9 Systemdesign

9.1 Hardwaredesign

9.1.1 Strøm sensor

Der anvendes 22 ohms specialmodstande, der kan håndtere 2 W, til alle rum. Dette gøres for at øge strømmen, der trækkes for hvert rum. Dette vælges, da strømmen ellers ville være så lav, at støj ville påvirke målingerne fra ACS712 i samme størrelsesorden som signalet, når rum er aktive.

Der anvendes 100 ohms modstande til hver LED, som også trækker en strøm (uddybning af strømberegning følger senere). Denne strøm er så relativt lille, at den ignoreres her. Der regnes derfor kun med strømmen gennem de 22 ohms modstande.

$$\frac{\mathbf{5V}}{\mathbf{22\Omega}} \approx \mathbf{227,27mA}$$

Valget giver omkring 227 mA pr. rum hvilket giver et signal, der ligger klart over sensorens støj-niveau og er dermed egnet til stabil detektion.

For ACS712 (5A) er følsomheden typisk 185 mV/A[3], hvilket giver et tydeligt spændingsudslag pr. aktivt rum:

$$V_{out} = 0.32A \cdot 185\frac{mV}{A} = 59.2mV$$

$$\boldsymbol{V_{out} = 227,27mA \cdot 185\frac{mV}{A} \approx 42,05mV}$$

Sensorens støj ligger omkring 21 mV peak-to-peak[4], så signalet fra ét rum ligger markant over støjgulvet:

$$\boldsymbol{\Delta V_{step} = 42,05mV > 21mV\ (støj)}$$

Dette gør det muligt at skelne mellem 1, 2 og 3 aktive rum med højere robusthed mod tilfældige udsving i målingen fra støj.

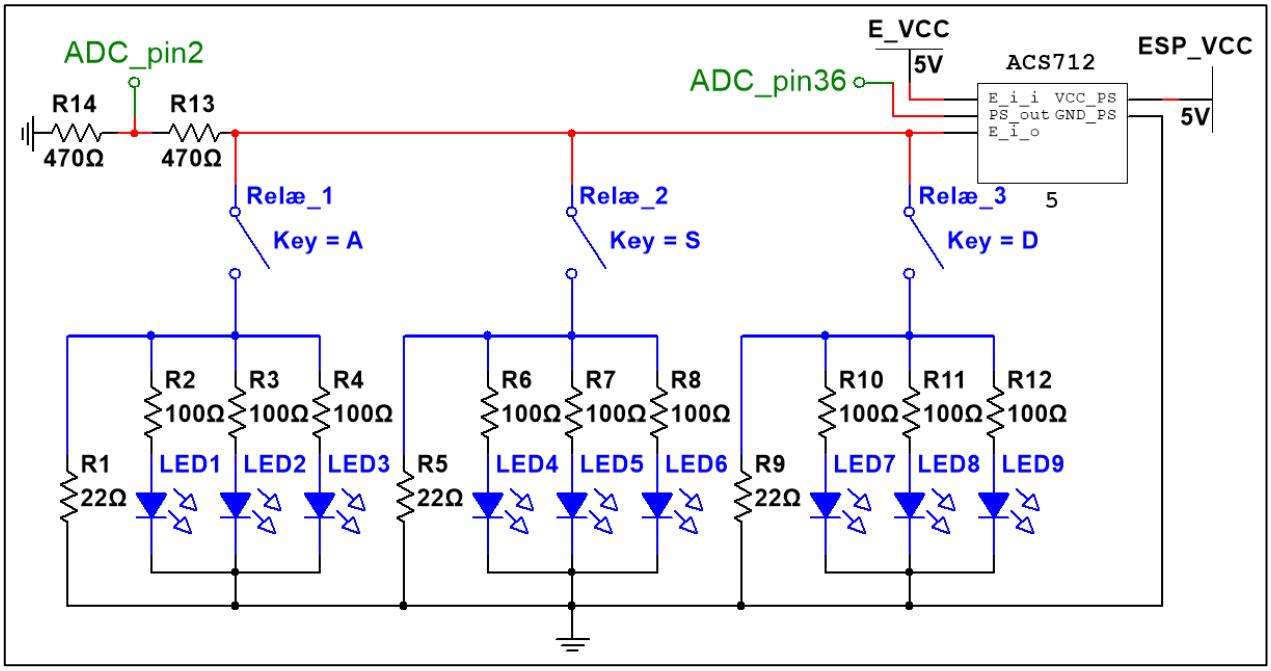
Det samlede strømforbrug af 22 ohm modstanden ved fuld belastning:

$$\boldsymbol{i_{total} = 3 \cdot 227mA = 681\ mA}$$

[3]Bilag 1.F Datablade i "ACS712 Strømsensor datablad"
[4]Bilag 1.F Datablade i "ACS712 Strømsensor datablad"

Der vil ofte være en situation, hvor ingen rum bør være aktive i prototypen. Derfor vil der ikke løbe nogen strøm. Dette vil med stor sandsynlighed aldrig ske i den virkelige verden, da der altid vil være aktivitet i minimum et rum på f.eks. et hospital.

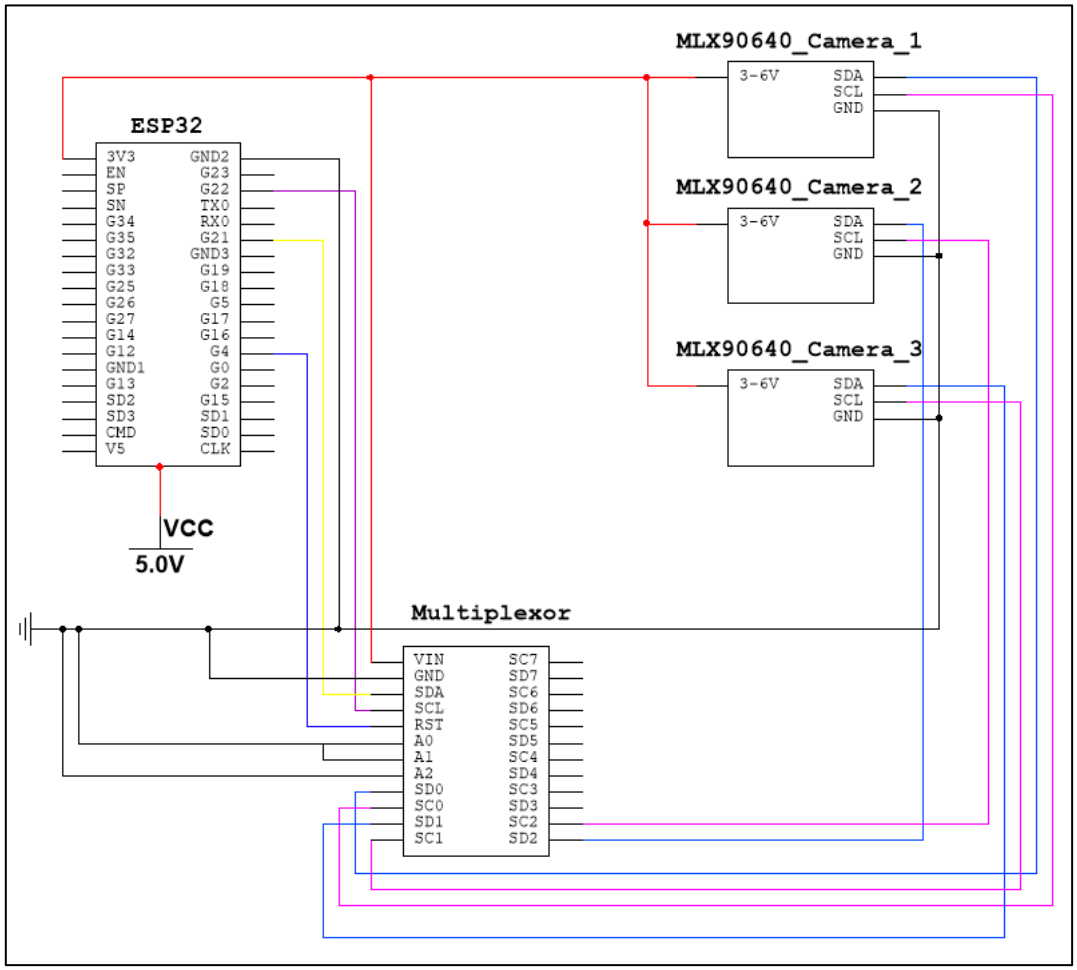
Derfor kunne strøm sensoren ikke anvendes til at tjekke om PMS bør være aktiv, da den ikke kan kende forskel på ingen aktive rum og ingen strøm fra Emergency Power. Derfor bruges ADC_pin2 til at tjekke om der er spænding fra spændingskilden. Og dermed undersøges om der er aktivitet fra Emergency Power.
Samlet tegning for strøm sensor opbygning ses på figur 25. Relæ er lavet til switches for simpelhedens skyld. For mere detaljeret tegning af disse se "LED lamper og Relæ" afsnit.



Figur 25 - Multisim tegning af nødvendigt kredsløb for strøm sensor

9.1.2 IR sensorer og Multiplexer

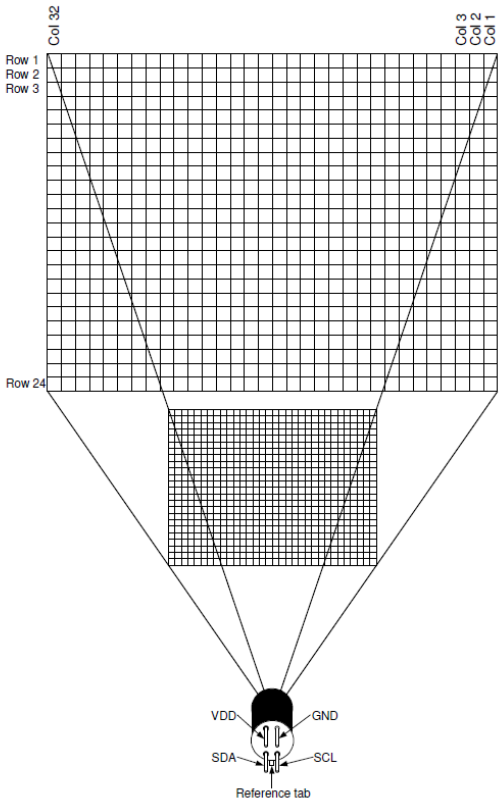
Til detektering af tilstedeværelse i de tre separate rum benyttes tre MLX90640 Thermal Cameras. Der bruges 2 IR sensorer, som overvåger i en 55° vinkel og 1 IR sensor, som overvåger i en 110° vinkel, da der ikke var 3 ens IR sensorer af denne model tilgængeligt. Da disse kameraer kommunikerer via I²C kommunikation, er det netop på denne måde, ESP32 kommunikerer med IR sensorerne. Da ESP32 kun har pins til I²C kommunikation med 1 komponent, er det nødvendigt at implementere en PCA9548A multiplexer. Multiplexeren fungerer som en 1-til-8 bidirektionel switch, der gør det muligt for ESP32'eren at kommunikere med hver sensor individuelt ved at skifte mellem de forskellige I²C-kanaler via pins SD0/SC0 til SD2/SC2. Da alle tre sensorer har identiske I²C-adresser, er det desuden muligt på denne måde at skelne mellem sensorerne.



Figur 26 - Diagram over forbindelserne mellem ESP32, Multiplexer samt IR sensorerne.

Som det fremgår af kredsløbsdiagrammet på figur 26, er multiplexerens adresseben A0-A2 trukket til jord, hvilket resulterer i en fast kontroladresse på 0x70.[5]

Det bør noteres, at der flere gange i løbet af rapporten bruges ord som ”frame”, ”background-frame” eller lignende. Disse udtryk henviser til den ramme af pixels, som sensoren måler i, hvilket fremgår af figur 27.[6]



Figur 27 - Pixel position i et MLX90640 Thermal Camera.

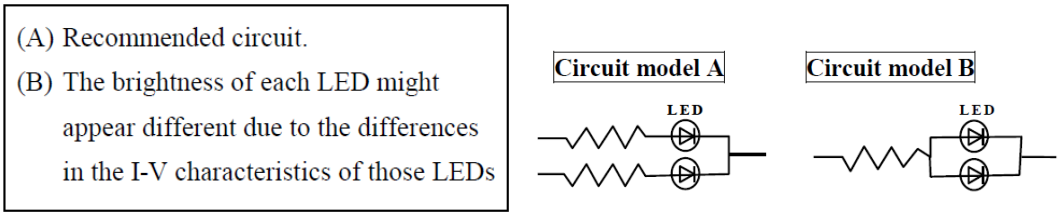
[5] Bilag 1.F Datablade i ”I2C multiplexer_tca9548a”, side 14, Table 1. Address Reference.
[6] Bilag 1.F Datablade i ”IR Sensor MLX90640-Datasheet-Melexis”, side 10

9.1.3 LED lamper og Relæ

9.1.3.1 LED lamper

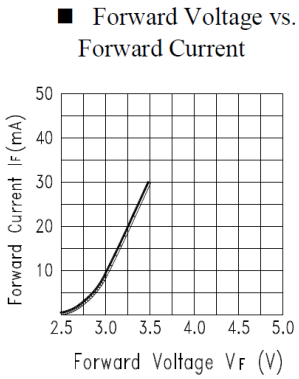
LED-indikatorerne anvendes som visuel demonstration af strømfordelingen. Hver LED-indikator repræsenterer strømforsyningen til et rum, hvilket gør det muligt at observere PMS-enhedens beslutninger i praksis, når den åbner eller lukker for strømkredsløb.

LED'erne er sat i parallelforbindelse, idet det sikrer, at LED'erne ikke er afhængige af hinanden. Det betyder, at hvis én af LED'erne ikke virker, så vil de resterende LED'er være upåvirkede og lyse fortsat. Derudover anbefaler producenten af LED-modellen at benytte netop denne form for kredsløb, hvilket kan ses nedenfor på figur 28.[7]



Figur 28 - Forslåede kredsløb til LED'er fra databladet

For at kunne udregne hvilken formodstand der skal anvendes til hver LED i lamperne, benyttes databladet for anvendte LED'er til at finde grafen for "Forward Voltage"[8], som skal bruges i udregningen af formodstande.



Figur 29 - Forward Voltage for LTW-2S3D8 fra datablad

$$R = \frac{V_{cc} - V_f}{I_{LED}}$$

V_{cc} = 5V fra spændingsforsyningen
V_f = 3.2V LED'ernes fremadspænding
I_{LED} = 0.018A Ønsket strømstyrke

[7] Bilag 1.F - Datablade i "LED LTW-2S3D8", side 9
[8] Bilag 1.F - Datablade i "LED LTW-2S3D8", side 4

$$\mathbf{100\Omega} = \frac{\mathbf{5V - 3.2V}}{\mathbf{0.018A}}$$

Fra ovenstående afsnit "Strøm Sensor" anvendes der en power resistor på 22Ω som sidder parallelt over de 3 LED'er og tilhørende 3 modstande på 100Ω.

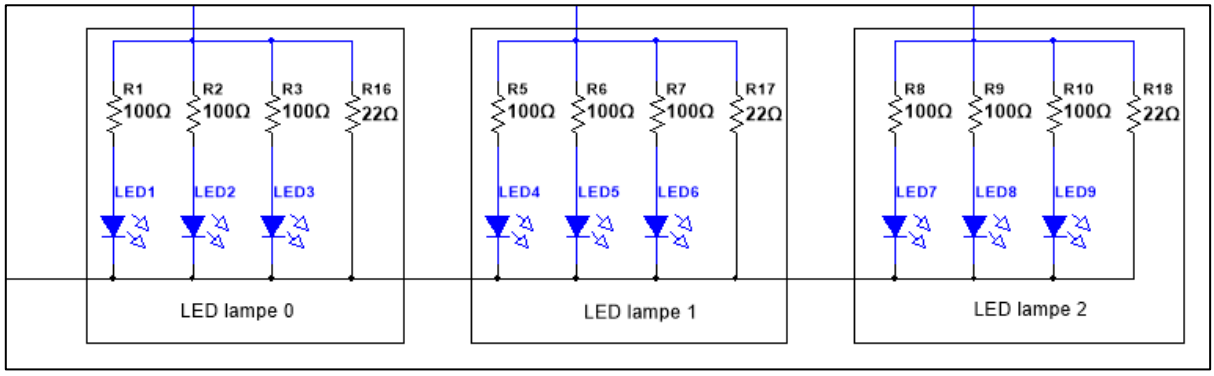
$$\boldsymbol{I_{LEDs} = 3 \cdot 18mA = 54mA}$$

Strømmen gennem power resistor beregnes til:

$$\boldsymbol{I_{PR} = \frac{5V}{22\Omega} \approx 227mA}$$

Den samlede strøm beregnes til:

$$\boldsymbol{I_{total} = I_{LEDs} + I_{PR} = 54mA + 227mA = 281mA}$$



Figur 30 - Multisimdiagram af LED-lamper

9.1.3.2 Relæsystem

Relæsystemet fungerer som broen mellem den digitale styring ESP32 og selve effektdistributionen fra Emergency Power.

DC coil data - 0.5 W sensitive (types 40.31/51/52/61/62)

Nominal voltage	Coil code	Operating range		Resistance	Rated coil consumption
U_N		U_{min}*	U_{max}	R	I at U_N
V		V	V	Ω	mA
5	**7**.005	3.7	7.5	50	100

Figur 31 - Specifikationer for spolen i relæet

Fra databladet for Finder relæet 40.52.7.005.0000 aflæses ved en forsyning på 5V (*Nominal voltage*), at der er en indre modstand på 50Ω og dermed en *Rated coil consumption* på 100 mA gennem spolen for at aktivere relæet.[9]

Ifølge databladet for ESP32 kan dens pins levere mellem 20mA og 40mA.[10] Dermed kan en ESP32 ikke levere den nødvendige strøm på 100 mA direkte fra en GPIO-pin til at aktivere relæet, og der er derfor implementeret et driver-kredsløb baseret på en BSX20 NPN-transistor. For en yderligere begrundelse for valg af disse hardwarekomponenter henvises der til Bilag 1.A - Analyse "Valg af yderligere hardware komponenter".

Transistoren skal mættes helt for at fungere som en effektiv kontakt.

$$\boldsymbol{R_B = \frac{V_{ESP} - V_{BE}}{I_B}}$$

V_{ESP} = 3.3V strøm fra en logisk GPIO pin G17 fra ESP32
V_{BE} = 0.7V generel værdi af spændingsfald over transistoren mellem base og emitter
I_B = strøm ind i basen af transistoren

Base Emitter Saturation Voltage	$V_{BE\ (Sat)}$*	I_C = 10mA, I_B = 1mA	0.7	0.85
		I_C = 100mA, I_B = 10mA	-	1.5

Figur 32 - Base Emitter Saturation Voltage sammenhæng fra datablad for transistor BSX20

Collectorstrømmen er aflæst i databladet til 100mA, og derfor bliver basisstrømmen 10mA, hvilket kan konkluderes ud fra figur 32.[11] Beregningen for basismodstanden bliver dermed følgende:

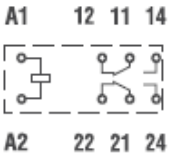
$$\boldsymbol{260\ \Omega = \frac{3.3V - 0.7V}{10mA}}$$

[9] Bilag 1.F - Datablade i "Relæ datablad", side 10
[10] Bilag 1.F - Datablade i "ESP32-wroom-32_datasheet_en", side 26
[11] Bilag 1.F - Datablade i "Transistor BSX20 - 2864188", side 2

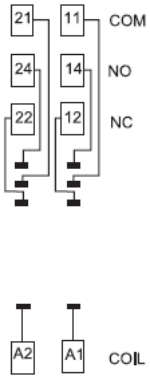
Transistoren fungerer som en elektronisk kontakt, der styres af et 3.3V signal fra ESP32 gennem den udregnede 260 Ω basismodstand. Denne modstandsværdi sikrer, at transistoren mættes (saturation), hvilket minimerer spændingsfaldet over transistoren og sikrer, at Finder 40.52-relæet modtager den fulde aktiveringsspænding på 5V.

For at beskytte styringselektronikken mod de induktive spændingsspidser, der opstår, når relæspolen deaktiveres, er der monteret en flyback-diode 1N4007 parallelt over relæet. Denne er valgt, fordi den kan holde til en spændingsspids på over 5V, som er den spænding, der skal til for at aktivere relæet. Flyback-diode 1N4007 kan håndtere en maksimal spændingsspids på 1000V[12], og dermed sikrer den, at transistoren ikke brændes af pga. en eventuel spændingsspids. Disse kunne forekomme ved switch i relæet, hvor spændingen går fra 5V til 0V.

Der bruges Active-LOW logik for relæene, idet det er meningen, at alle rummene skal være tilført strøm i det øjeblik blackout finder sted og PMS aktiveres. Når systemet er startet op, skal prioriteringen derefter tage over og strømmen til rummene tændes eller slukkes afhængigt af denne prioritering.



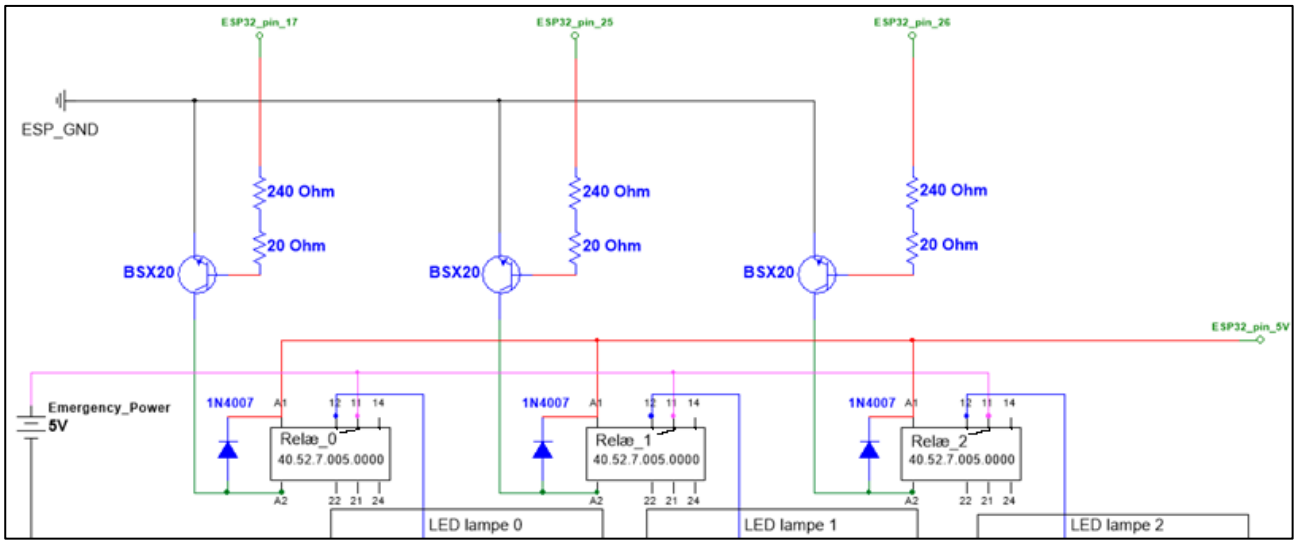
Figur 34 - Diagram over relæ 40.52.7.005.0000 set nedefra.



Figur 33 - Diagram over relæ 40.52.7.005.0000 med tekst, der angiver, hvad tallene betyder.

Ben 12 på relæet hedder "NC" ("Normally Closed"), hvilket betyder, at dette ben leder strømmen videre, når relæet ikke er aktiveret.[13] Derfor er ben 12 forbundet til LED'erne, og på den måde opnås en Active-LOW logik.[14]

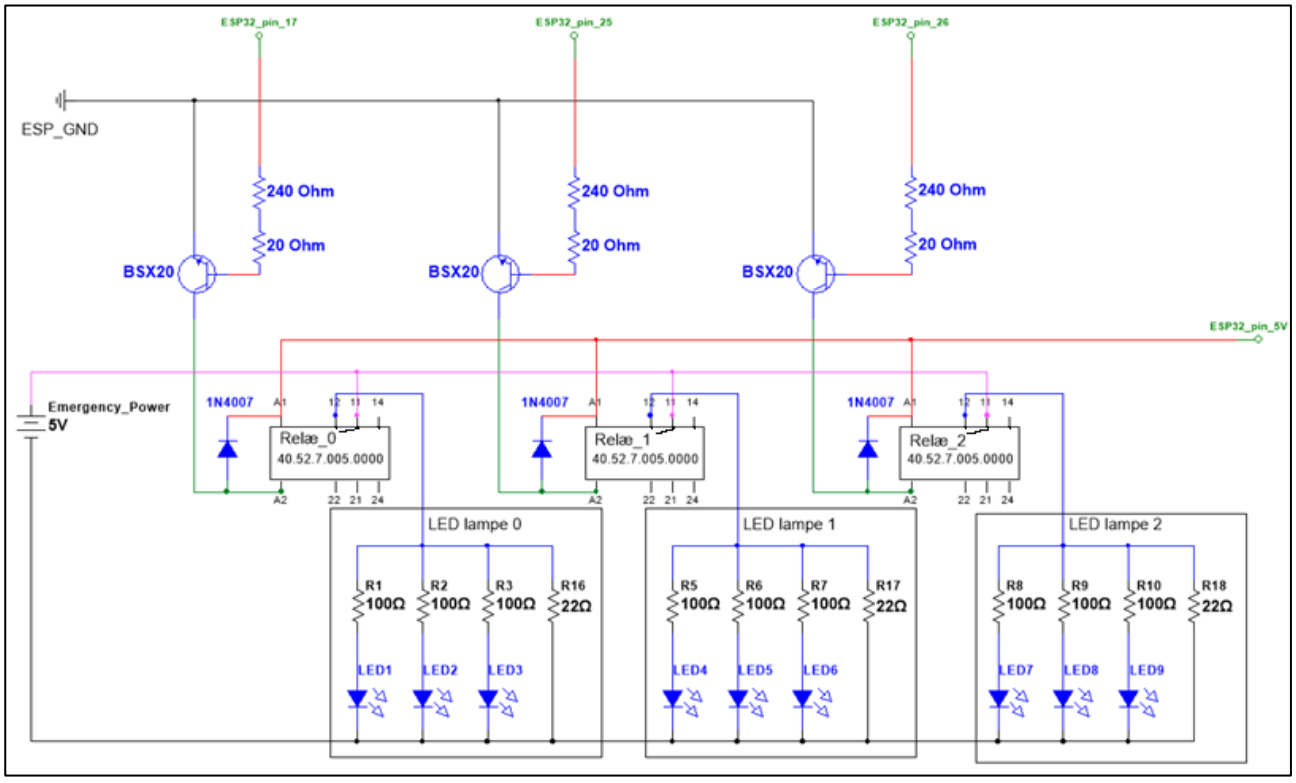
[12] Bilag 1.F - Datablade i "Diode_til_relæ_1n4007"
[13] Bilag 1.F - Datablade "Relæ datablad", side 3.
[14] Bilag 1.F - Datablade "Relæ datablad", side 14 under sokkel 95.P5



Figur 35 - Multisimdiagram af Relæsystem

9.1.3.3 Samlet Relæsystem og LED-lamper

Figur 36 viser diagramsammensætningen af de 3 relæsystemer og de 3 LED lamper.



Figur 36 - Relæsystem tilkoblet LED lamper

9.2 Softwaredesign

9.2.1 Power Sensor

Mark

Overblik over power sensor klasse, herunder metoder og attributter, kan ses på figur 37.

```
class PowerSensor {
public:
  PowerSensor(double capacity_);
  bool unitTestPowerSensorLower();
  bool unitTestPowerSensorUpper();
  double getPowerDraw();
  void updateCapacity();
  double getCapacityPct();
  bool emergencyPowerNeeded();
  void resetCapacity();

private:
  double capacity_;
  int lastUpdateMs_;
  double startCapacity_;
  double capacityPct_;
};
```

Figur 37 - PowerSensor klasse oversigt

Attribut beskrivelse:

Capacity_: Start kapaciteten af det hypotetiske batteri. Gemmes i milliampere-timer, hvilket bruges til at simulere et batteri af den størrelse (mAh).

lastUpdateMs_: Bruges til at holde styr på hvor lang tid der er gået siden sidste *capacity_* opdatering og anvendes til at oversætte en strøm måling (mA) til en strøm måling over tid (mAh). Denne omregning forklares i metode afsnit (updateCapacity()).

startCapacity_: Bruges til at holde styr på hvor stor kapacitet der var ved system start, før der er blevet trukket noget strøm.

capacityPct_: Fortæller hvor stor procentdel af den originale strøm kapacitet er tilbage i batteriet.

Klassen PowerSensors namespace:

Der er en del data som bliver genbrugt igennem en række PowerSensor metoder. Derfor anvendes et namespace til at holde styr på dette data, og for at øge læsbarheden i metoderne.

```
namespace {
static int kAdcZero = 3040; // Updateres i unit test lower. Værdi hvor maks 3% ADC målinger ryger over ved ingen rum aktive.
static int kAdcFull = 3170; // Updateres i unit test upper. Værdi hvor maks 3% ADC målinger falder under ved alle rum aktive
constexpr int kFullScale_mA = 800; // Hard coded udfra multisim mA måling.
constexpr int kSamples = 20; // Antal målinger avg ADC beregninger
constexpr double kMsPerHour = 3600000.0;
}
```

Figur 38 - PowerSensor.cpps namespace

Forklaring af namespaces data:

kAdcZero: Er den ADC-måling der forventes, når der ikke trækkes strøm. Er sat til 3040 fra start. Gættet kommer på baggrund af data opsamlet med ESP32 og strømsensor alene i lille isoleret system. Dette data kan ses i Bilag 1.E - PowerSensor ”Data taget med ESP32 til Power Sensor kalibration”. *kAdcZero* er static, da den skal opdatere metoderne unitTestPowerSensorUpper() & unitTestPowerSensorLower(). Static gør, at *kAdcZero* kun er synlig i powerSensor.cpp og ikke globalt.

kAdcFull: Er den ADC-måling der forventes, når der trækkes maksimal strøm (tre aktive rum). Data til gæt kan findes i Bilag 1.C - Design, der refereres i *kAdcZero*. Static af samme grund som *kAdcZero*.

kFullScale_mA: Er det maksimale strøm der kan trækkes i systemet (tre aktive rum). Set på spændingskilde i lab.

kSamples: Er det antal målinger der tages, ved ADC-målinger. Der tages altid mere end en måling, og tager et gennemsnit af *kSamples*. Dette gøres for at øge præcision og mindske støjs virkning.

kMsPerHour: Er antal millisekunder der går på en time. Bruges i metoden: updateCapacity().

PowerSensors metoder:
bool unitTestPowerSensorLower()

```
bool PowerSensor::unitTestPowerSensorLower() { // tester ADC læsning ved ingen rum aktive.
  int lowerBoundary{3000}; // gæt på værdi er markant under maksimale ADC måling.
  int attempts{0};

  // Slukker alle rum, ved at aktivere
  // Skal bruges til slukke PMS, når strøm net er tilbage
  pinMode(17, OUTPUT);
  pinMode(25, OUTPUT);
  pinMode(26, OUTPUT);
  digitalWrite(17, HIGH);
  digitalWrite(25, HIGH);
  digitalWrite(26, HIGH);

  delay(3000); // Delay så strøm når at falde i systemet.
```

Figur 39 - Udsnit af source kode til unitTestPowerSensorLower()

Til start sættes *pinMode* pins 17, 25 & 26 til OUTPUT og pins sættes til HIGH, som slukker for lys til alle rum (ingen strøm). Samt der bliver lavet et par integers som bruges igennem metodens forløb. Så ventes der 3 sekunder for at sørge for, at strømmen er stabiliseret.
Metodens opgave er at finde en passende *lowerBoundary* værdi. En passende værdi er defineret ved, at maksimalt 3% af målinger overstiger værdien. På den måde fås en værdi, hvor der med 97% sikkerhed kan sige at et rum ikke er aktiv, da kun 3% af målinger uden strøm overstiger værdien.

```
  while(1) {
    int outliers{0};
    for (int i = 0; i < 200; i++) {
      int sum{0};
      for (int j = 0; j < kSamples; j++) {
        sum += analogRead(36);
      }
      int avg = sum / kSamples;
      if(avg > lowerBoundary) {
        outliers++;
      }
    }

    if (outliers >= 6) {
      attempts++;
      lowerBoundary++;
    } else {
      kAdcZero = lowerBoundary;
      return true;
    }
    if (attempts >= 100) { // Fail safe
      return false;
    }
  }
}
```

Figur 40- Udsnit af source kode til unitTestPowerSensorLower()

Metoden tager 200 målinger af gennemsnitsværdier af *kSamples*. Der tælles outliers hver gang en af målingerne af *kSamples* overstiger *lowerBoundary* værdi. Hvis der er der mindre end 6 målinger der overstiger *lowerBoundary* værdi sættes *kAdcZero* til sættes den til *lowerBoundarys* værdi. Og der returneres true, som betyder at kalibrering er sket uden fejl. Hvis der bruges over 100 forsøg på at finde en værdi, så returneres false, som betyder kalibrering ikke lykkes. Og det tyder på der er en sensor fejl.

bool unitTestPowerSensorUpper():
I grove træk det samme som unitTestPowerSensorLower(). Beskrivelse af metoden findes i Bilag 1.C - Design, som henvises til i slutningen af metodebeskrivelsen.

double getPowerDraw():
Metoden laver et bud på hvor stor strøm træk der er i systemet. Den bruger *kAdcZero* og *kAdcFull* til disse beregninger. Metoden laver følgende beregning, Summer *kSamples* målinger sammen:

$$sum = \sum_{i=1}^{kSamples} ADC_i$$

```
double PowerSensor::getPowerDraw() {
  int sum = 0;
  for (int i = 0; i < kSamples; i++) {
    sum += analogRead(36);
  }
```

Figur 41 - Udsnit af getPowerDraw source kode

Gennemsnit af værdi af *kSamples* ADC-målinger:

$$avg = \frac{sum}{kSamples}$$

```
int avg = sum / kSamples;
if (avg <= kAdcZero) {
  return 0;
}
```

Figur 42 - Udsnit af getPowerDraw source kode

Span forskelle mellem *kAdcFull* og *kAdcZero*:

$$\boldsymbol{span = kAdcFull - kAdcZero}$$

```
int span = kAdcFull - kAdcZero;
if (span <= 0) {
  return 0;
}
```

Figur 43 - Udsnit af getPowerDraw source kode

Hvis avg er under *kAdcZero* (beregnet efter ikke aktive rum), så returneres 0, altså der trækkes ingen strøm ellers fra regnes *kAdcZero* fra avg:

$$\Delta = \boldsymbol{avg - kAdcZero}$$

```
double delta = avg - kAdcZero;
```

Figur 44- Udsnit af getPowerDraw source kode

Konvertere til estimeret strøm i mA:

$$\boldsymbol{I_{mA} = \frac{\Delta \cdot 800}{span}}$$

```
  double current_mA = (delta * kFullScale_mA) / span;
  if (current_mA < 0) {
    current_mA = 0;
  }
  if (current_mA > kFullScale_mA) {
    current_mA = kFullScale_mA;
  }

  return current_mA;
}
```

Figur 45 - Udsnit af getPowerDraw source kode

Begrænset til minimum 0 og maksimum 800. 800 er for de 800mA (*kFullScale_mA*) systemet kan trække.
Det er denne værdi som der bliver returneret, medmindre den falder under nul. Hvis det er tilfældet, så returnes 0.

void updateCapacity():

```
void PowerSensor::updateCapacity() {
  double now = millis(); // millis returnere antal milisekunder siden programmet startede.
  double elapsedMs = now - lastUpdateMs_;
  lastUpdateMs_ = now;

  double current_mA = getPowerDraw();
  double draw = (double)current_mA * (double)elapsedMs / kMsPerHour;

  if (capacity_ <= 0) {
    return;
  }

  if (draw >= capacity_) {
    capacity_ = 0.0;
    return;
  }
  capacity_ -= draw;
  capacityPct_ = capacity_/startCapacity_*100;
}
```

Figur 46 - Source kode til updateCapacity()

Metodens opgave er at oversætte mA måling fra getPowerDraw() til en mAh.
Beregning til dette:
Strømforbrug over siden sidste måling (mAh).

$$\boldsymbol{draw_{mAh}} = \frac{\boldsymbol{I_{mA}} \cdot \boldsymbol{\Delta t_{ms}}}{\mathbf{3600000}}$$

Metoden antager at state (antal aktive rum og dermed powerdraw) er uændret siden sidste opdatering. Da metoden kun tager et øjebliks billede med getPowerDraw(). Men der opdateres også med relativt lille mellemrum (lidt over 200ms), så det passer nogenlunde.

Hvis kapaciteten er under 0 eller lig nul, så gør metoden intet.

Hvis draw er større end eller lig med kapacitet, så bliver den sat til nul. På den måde undgås det at kapaciteten ryger under 0.

Til slut opdaterer metoden *capacity_* og *capacityPct_*:

$$\mathbf{capacityPct_} = \frac{\mathbf{capacity_}}{\mathbf{startCapacity_}} \cdot \mathbf{100}$$

Beskrivelser af de resterne metoder kan findes i Bilag 1.C - Design.

9.2.2 IR Sensor

Koden for de tre MLX90640 Thermal Cameras er skrevet i Arduino IDE 2.3.9, som essentielt fungerer som en ”wrapper” for C++ kode.

For at lave softwaren til IR sensorerne er der udarbejdet klassen IRsensor, som indeholder både en række funktioner og attributter. Funktionerne er deklareret i IRsensor.h filen, og implementeringen af disse samt de private attributter kan findes i IRsensor.cpp filen.

Nedarvning

IRsensor nedarver fra ”AdaFruit_MLX90640”[15]. Det betyder, at et IRsensor objekt er relateret til ”Adafruit” biblioteket og kan bruge dets indbyggede funktioner.

```
class IRsensor : public Adafruit_MLX90640 {
```

Figur 47 - Denne linje kode viser, at IRsensor klassen er en nedarvning fra Adafruit_MLX90640 klassen. Se linje 6 i IRsensor.h filen.

Da IRsensor klassen skal have adgang til en række funktioner fra det offentlige Adafruit_MLX90640 bibliotek, er IRsensor klassen en nedarvning af dette bibliotek, hvilket også kan ses af klassediagrammet via relationen mellem IRsensor klassen og Adafruit biblioteket, se figur 47 og figur 8. Yderligere er 3 funktioner tilføjet IRsensor klassen, som er følgende:

- IRsensor()
- void init(int id, float* bgFrame)
- bool getActivity(float* current, bool initOn)

IRsensor() konstruktør

```
IRsensor::IRsensor() {};
```

Figur 48 - IRsensor konstruktøren implementeret i IRsensor.cpp filen. Se linje 4 i IRsensor.cpp filen.

I softwaredesignet til IR-sensorer bruges der for-løkker til at gennemgå IRsensor-objekterne. Dette er for at undgå spaghettikode, da der ikke skal skrives præcis samme kode for hver af de 3 IRsensor objekter. Dette betyder også, at der laves et array af 3 IRsensorer, hvortil der bruges den tomme IRsensor-konstruktør. Dette sker dog inde i InitPMS.h filen, da der er her initieringen finder sted.

```
IRsensor sensor[3];
```

Figur 49 - Oprettelse af et array med 3 IRsensor objekter. Se linje 52 i InitPMS.h filen.

[15] Eksternt bibliotek: ”Adafruit_MLX90640” - Link: https://github.com/adafruit/Adafruit_MLX90640.git

Konstruktøren er tom, fordi IRsensor objekterne først initieres senere via init-funktionen. IR-sensor objekterne skal bruge en *backgroundFrame*, som de får fra InitPMS, og denne kobling sker først i setupHardware-funktionen, som ikke har fundet sted på det tidspunkt, hvor IRsensor objektet i main er oprettet. Derfor er det vigtigt, at konstruktøren er tom, så objektet ikke leder efter en *backgroundFrame*, som endnu ikke er blevet sat op. Det ville dog være muligt at oprette IRsensor objektet i main efter denne funktion er kørt, men i så fald ville det kræve et mere omfattende brug af pointere og dynamisk allokering af hukommelse, hvilket gør koden mere kompliceret end nødvendigt.

IRsensor objektet oprettet i main er yderst nødvendigt og er oprettet separat fra de 3 IRsensor objekter i arrayet fra InitPMS.h filen, idet dette objekt bruges som kameraet, der tager et billede i hvert rum i realtid. Inde i hvert IRsensor objekt i arrayet ligger baggrundsbilledet fra hvert rum.

void init(int id, float* bgFrame)

```
void IRsensor::init(int id, float* bgFrame) {
  id_ = id;
  backgroundFrame_ = bgFrame;
}
```

Figur 50 - void init(int id, float bgFrame) funktionen, der initierer objekter af IRsensor klassen. Se linje 7-10 i IRsensor.cpp filen.*

Denne funktion bruges til at initiere IRsensor objekterne. Den tager parametrene *id*, som er data af typen int, samt *bgFrame*, som er en pointer, der peger på data af typen float. Funktionen returnerer intet, og returværdien er derfor void. I stedet udfører funktionen 2 assignments; den private attribut *id_* får værdien *id*, og den også private attribut *backgroundFrame_* tildeles hukommelsesadressen af pointeren *bgFrame* fra parametrene. Bevidst vælges der at bruge en pointer i dette tilfælde, fordi det betyder at, når der via hver IR sensor tjekkes, hvorvidt der er aktivitet eller ej, så bruges pointeren til at pege på den pågældende IR sensors baggrundsbillede. Dermed indeholder hvert IRsensor objekt et id og et baggrundsbillede af det rum, IR-sensoren er i. Baggrundsbilledet kommer fra kalibreringen af IR sensorerne, hvilket uddybes nærmere i afsnittet ”InitPMS” under afsnittet Softwaredesign afsnit 9.2.5. Det er vigtigt, at hvert IRsensor objekt indeholder et baggrundsbillede fra kalibreringen, idet det gør det muligt at skelne mellem mennesker og varme objekter, som konstant er i det pågældende rum.

bool getActivity(float* current, bool initOn)

```
bool IRsensor::getActivity(float* current, bool initOn) {
    if (initOn) {
      int count = 0;
      for (int y = 1; y < 23; y++) {
          for (int x = 1; x < 31; x++) {
            int p = y * 32 + x;
            if ((current[p] - backgroundFrame_[p]) > TEMP_THRESHOLD && current[p] > MIN_HUMAN_TEMP) {
                int neighbors = 0;
                if ((current[p-1] - backgroundFrame_[p-1]) > (TEMP_THRESHOLD * 0.8)) neighbors++;
                if ((current[p+1] - backgroundFrame_[p+1]) > (TEMP_THRESHOLD * 0.8)) neighbors++;
                if ((current[p-32] - backgroundFrame_[p-32]) > (TEMP_THRESHOLD * 0.8)) neighbors++;
                if ((current[p+32] - backgroundFrame_[p+32]) > (TEMP_THRESHOLD * 0.8)) neighbors++;

                if (neighbors >= 2) count++;
            }
          }
      }
      activity_ = (count >= MIN_CLUSTER_SIZE);
      return activity_;

    } else {

      return false;

    }
  }
```

Figur 51 - bool getActivity(float current, bool initOn) funktionen, som vurderer hvorvidt der er en person i det pågældende rum. Se linje 13-38 i IRsensor.cpp filen.*

getActivity-funktionen returnerer en bool, som fortæller, hvorvidt der er aktivitet i det pågældende rum eller ej. Funktionen tager 2 parametre; *current* som er en pointer, der peger på data af typen float, samt *initOn*, som er data af typen bool. Sidstnævnte parameter bruges til at tjekke, om initieringen i InitPMS har fundet sted, da IR sensorerne kun skal bruge funktionen, hvis initieringen er sket. Pointeren *current* peger på den *frame*, som IRsensor objektet fra main-filen indeholder. For at tjekke om der er en person i et rum, sammenlignes den nuværende *frame*, som er fra IRsensor objektet i main-filen, med den *backgroundframe*, som IRsensor objekterne fra arrayet fra InitPMS.h indeholder.

Hvis temperaturen af en pixel i IR sensorens *frame*, der viser miljøet i realtid, overstiger et threshold på 30 grader celsius, og hvis ændringen mellem den nuværende temperatur og baggrundstemperaturen overstiger 3 grader celsius, anses denne pixel for at have temperaturen af et menneske. Hvis denne pixel har mindst 2 naboliggende pixels omkring sig, som også opfattes for at vise temperaturen af et menneske, tælles denne pixel med i en variabel kaldet *count*, som kun indeholder heltal. Dog tjekkes ikke temperaturen i de yderste kolonner og yderste rækker, da disse ikke har alle nabopixels omkring dem. Nabopixels behøver blot at være 80% af *TEMP_THRESHOLD* på 3 grader celsius, fordi kernen i et menneske oftest er varmere end de ydre dele af et menneske. Den private attribut *activity_* returneres til sidst, og

hvis *count* er lig eller overstiger 12 pixels, vil *activity_* være ”True” og dermed angive, at der er aktivitet i rummet. For yderligere uddybning af softwaredesignet af IR sensorerne, se Bilag 1.C - Design.

9.2.3 GUI

GUI’en er opbygget i Python med bibliotekerne Tkinter[16], Tkinter.ttk[17], serial[18] og threading[19].

GUI’en består af mange moduler og funktioner, se evt. kode i Bilag 1.C - Design. Kort opsummeret så kreeres og destrueres hver side hver gang man skifter imellem de forskellige menuer, for at kunne lave sideskift. Vigtigt data gemmes i globale variabler, sådan at de kan genindlæse, ved hver kreation af en side der anvender dataet. For videre uddybelse se Bilag 1.C - Design.

Kommunikation imellem GUI’en lokaliseret på Raspberry Pi og logikken lokaliseret på ESP32-WROOM, anvendes seriel kommunikation, og til dette har GUI’en to funktioner til henholdsvis at læse og modtage data. Designet af disse uddybes i sektionerne nedenfor.

9.2.3.1 Read_serial()

Til at kommunikere imellem ESP32-WROOM og Raspberry Pi anvendes en seriel driver read_serial() som både modtager dataet fra ESP32’en og behandler det modtaget data så der eksekveres de nødvendige funktioner, baseret på dataet fra ESP32’en.

For at undgå at eventuelle fejl/exceptions crasher hele programmet anvendes try og except, så hvis der sker en fejl, fortsætter programmet med at køre. Inde i try læses dataet der modtages fra ESP32 ind i variablen *line* som så behandles forskelligt alt efter om det er en talværdi, eller noget andet der bliver sendt. Hvis det er en talværdi der bliver modtaget opdateres power baren på forsiden med den talværdi der er modtaget fra ESP32’en.

Hvis der modtages en anden type data der passer med en af de fire betingelser “ok”, “fejl”, “idle” eller “sensor” eksekveres den tilhørende betingelse så den passende popup popper op. “ok” og “fejl” eksekverer den samme popup funktion, men med True/False til forskel, som også sættes, ud fra hvilken popup der skal eksekveres. “idle” og “sensor” eksekverer forskellige funktioner der passer til den besked der skal sendes til User.

Ved både talværdier og andre værdier kaldes den ønskede funktion med Init.root.after(0, funktion), hvor kun funktionen er til forskel mellem kaldene. Hvis det er en talværdi, sendes både funktionskaldet og en int værdi med.
Init.root er det vindue som eksekveringen skal ske på, og *.after(0*, betyder at eksekveringen

[16] Eksternt bibliotek: ”Tkinter” - Link: https://github.com/python/cpython/tree/3.14/Lib/tkinter
[17] Eksternt bibliotek: ”Tkinter.ttk” - Link: https://github.com/python/cpython/blob/3.14/Lib/tkinter/ttk.py
[18] Eksternt bibliotek: ”serial” - Link: https://github.com/pyserial/pyserial/
[19] Eksternt bibliotek: ”threading” - Link: https://github.com/python/cpython/blob/3.14/Lib/threading.py

skal ske efter 0 millisekunder, altså med det samme. Derefter gives, den funktion som skal eksekveres, med som parameter. For behandlingen af talværdier, bliver funktionen også givet med parameteren *int(line)* som er den talværdi *update_power_callback* skal opdateres med.

Hvis dataet er hverken en talværdi eller én af de 4 strings, vil der ikke ske noget og hvis der bliver kastet en ”exception”, skal der ikke ske noget, programmet skal bare køre videre uden at behandle ”exception” da den kun er der, som nævnt tidligere, for at undgå at programmet crasher ved undtagelser.

```
def read_serial():

    while True:
        try:
            line = ser.readline().decode('utf-8').strip()      You, last week
            if line.isdigit():
                Init.root.after(0,home_page.update_power_callback, int(line))
            else:
                if line == "ok":
                    Popup.SuccessfulBoot = True
                    Init.root.after(0,Popup.SystemBootPopup)
                elif line == "fejl":
                    Popup.SuccessfulBoot = False
                    Init.root.after(0,Popup.SystemBootPopup)
                elif line == "idle":
                    Init.root.after(0,Popup.SystemShutdownPopup)
                elif line == "sensor":
                    Init.root.after(0, Popup.SensorFail)
        except:
            pass
```

Figur 52 - Source code til read_serial()

9.2.3.2 Send_serial()

Send_serial() funktionen har en parameter som her kaldes *data*, hvilket er det som skal sendes. Inde i funktionen køres et forsøg på at sende data ved at lægge dataet i en tom variabel message som er lig *data* og en enkelt ”newline” så data sendes på overskuelig vis. Derefter sendes beskeden med ser.write(), som er en funktion fra serial biblioteket som bliver inkluderet. Hvis ikke det er muligt at sende dataet, så sender den intet.

```
def send_serial(data):
    try:
        message = f"{data}\n"
        ser.write(message.encode('utf-8'))
        print("Sent:", message.strip())
    except:
        pass
```

Figur 53 - Source code til send_serial()

9.2.4 ProfileData

Mark

Klassen ProfileData bruges til at holde styr på hvilken ”profil” PMS anvender. Det vil sige, hvilken indstilling rummene styres efter. Oversigt over klassen kan ses på figur 54

```
class ProfileData {
public:
  ProfileData();
  void editProfile(int key, double threshold);
  double getThreshold(int key);

private:
    std::map<int, double> thresholds_;
};
```

Figur 54 - ProfileData klasse oversigt

Attribut beskrivelse:
Thresholds_: indeholder aktive thresholds af aktive profil. Der anvendes ”map” som ”container”, da logikken af at have en ”key”, som svarer til et rum nummer gjorde det nemt at tilkoble thresholds til rum.

ProfileDatas metoder:
Constructor:

```
ProfileData::ProfileData() : thresholds_{{0, 0},{1, 0},{2, 0}} {};
```

Figur 55 - Source kode til ProfileData constructor

Initierer thresholds for rum 0, 1 & 2 til en værdi på 0. Altså rum skal være aktive indtil der ikke er noget strøm tilbage.

void editProfile(int key, double threshold):

```
void ProfileData::editProfile(int key, double threshold) {
  thresholds_[key] = threshold;
}
```

Figur 56 - Source kode til editProfile()

Metoden tager en ”key” og en thresholds værdi. Den sætter så threshold-værdien af rummet (key) til threshold-værdien.

double getThreshold(int key):

```
double ProfileData::getThreshold(int key) {
  return thresholds_[key];
}
```

Figur 57 - Source kode til getThreshold()

Returnere threshold-værdien af ønsket rum (key)

9.2.5 InitPMS

Figur 58 viser de 4 metoder som samlet faciliterer evnen til at gemme og loade kalibreringerne for kameraerne.

```
9   class InitPMS {
10  private:
11      const char* CALIB_FILE = "/bg_kalibrering.bin";
12
13      bool loadBackground() {
14      if (!LittleFS.exists(CALIB_FILE)) {
15          return false;
16      }
17
18      File file = LittleFS.open(CALIB_FILE, FILE_READ);
19      if (!file) {
20          return false;
21      }
22
23      file.read((uint8_t*)backgroundFrames, sizeof(backgroundFrames));
24      file.close();
25      return true;
26      }
27
28
29      void saveBackground() {
30      File file = LittleFS.open(CALIB_FILE, FILE_WRITE);
31      if (!file) {
32          return;
33      }
34
35      file.write((uint8_t*)backgroundFrames, sizeof(backgroundFrames));
36      file.close();
37
38      }
```

Figur 58 - viser funktionen til at læse og gemme kalibreringen af de 3 kameraer.

Forklaring af attributer kan ses i Bilag 1.C – Design.

I figur 59 defineres metoden tcaselect(uint8_t i) som vælger, hvilken kanal der anvendes på Multiplexeren, der anvendes til at facilitere I²C kommunikation mellem MCU ESP32 og MLX90640 kameraerne. Den tjekker, hvorvidt inputtet til metoden ligger under 7, da Multiplexeren har 8 kanaler (0-7) og skriver herefter til Multiplexerens adresse på MCU ESP32, hvorefter den sætter den relevante kanal på Multiplexeren til input.

Herefter defineres metoden setInitOn(bool status) som sætter attributten *initOn_* til at være lig parameteren status. Den har returtypen void.

```
40  public:
41      const int RELAY_PINS[3] = {17, 25, 26};
42      float backgroundFrames[3][768];
43      bool isCalibrated[3] = {false, false, false};
44      bool systemReady = false;
45      bool initOn_ = false;
46
47      IRsensor sensor[3];
48
49
50
51
52  void tcaselect(uint8_t i) {
53      if (i > 7) return;
54      Wire.beginTransmission(0x70);
55      Wire.write(1 << i);
56      Wire.endTransmission();
57  }
58
59  void setInitOn(bool status) {
60      initOn_ = status;
61  }
```

Figur 59 - viser public attributter og metoder for InitPMS.

```
63  void setupHardware(IRsensor &sensoren, bool check) {
64      if (check && !initOn_) {
65          for (int i = 0; i < 3; i++) {
66              pinMode(RELAY_PINS[i], OUTPUT);
67              digitalWrite(RELAY_PINS[i], LOW);
68              Wire.begin(21, 22);
69              Wire.setClock(100000);
70              int fail_count = 0;
71              for (uint8_t i = 0; i < 3; i++) {
72                  tcaselect(i);
73                  delay(10);
74                  if (!sensoren.begin(MLX90640_I2CADDR_DEFAULT, &Wire)) {
75                      Serial.println("sensor");
76                      fail_count++;
77                  } else {
78                      sensoren.setMode(MLX90640_CHESS);
79                      sensoren.setRefreshRate(MLX90640_4_HZ);
80                  }
81              }
82              if(fail_count == 3) {
83                  systemReady = false;
84                  messageFail();
85                  while (1) {};
86              } else {
87                  systemReady = true;
88              }
89              for (int i = 0; i < 3; i++) {
90                  sensor[i].init(i, backgroundFrames[i]);
91              }
92              setInitOn(true);
93              systemReady = true;
94          }
95          if (systemReady) {
96              messageSuccess();
97          }
```

Figur 60 - viser metoden setupHardware() i klassen InitPMS.

Figur 60 viser metoden setupHardware().

Denne metode indstiller og initierer alle sensorer, LED relæ og Multiplexeren. Der køres et tjek om, hvorvidt PMS er initieret, og at der løber strøm fra Emergency Power. Hvis systemet ikke er initieret og Emergency Power løber, køres metoden. I et for-loop, som kører 3 iterationer,

sættes *RELAY_PINS* til at være output, og LED relæsættene sættes til LOW, som tænder for lysene i LED sættene.

Dette reflekterer, at når systemet tager over, skal alt strøm initialt løbe som normalt indtil, detektionslogikken aktiveres. Herefter aktiveres Multiplexerens SDA og SCL-forbindelser (Wire.begin(21, 22)). ESP32ens clock speed sættes til at være 100 kHz og *fail_count* variablen af typen int initieres til værdien 0. Herefter køres der endnu et for-loop i 3 iterationer som vælger hver af de 3 kameras kanaler i Multiplexeren gennem brug af metoden tcaselect(i). Herefter initieres hver sensor og *fail_count* inkrementeres for hver fejlet initialisering.

Hvis alle 3 sensorer fejler, sendes en fejl besked til GUIen, og systemet angiver, at initieringen fejlede. På anden vis kan kameraerne initieres gennem metoderne sensoren.setMode() og sensoren.setRefreshRate() til at køre i chess mode (pixels bliver arrangeret som et chess board) og ”refreshraten” sættes til 4 Hz. Yderligere indlæses *backgroundFrame* til de forskellige sensorer som deres kalibreringsbaggrund.

```
102  void calibrate(IRsensor &sensoren) {
103      if (Serial.available() > 0) {
104          String key = Serial.readString();
105          if (key == "reset\n" || key == "Reset\n" || key == "RESET\n") {
106              LittleFS.remove(CALIB_FILE);
107              isCalibrated[0] = false;
108              isCalibrated[1] = false;
109              isCalibrated[2] = false;
110          }
111      }
112          if (!LittleFS.begin(true)) {
113          } else {
114              if (loadBackground()) {
115              isCalibrated[0] = true;
116              isCalibrated[1] = true;
117              isCalibrated[2] = true;
118              }
119          }
120          float tempFrame[768];
121          for (uint8_t i = 0; i < 3; i++) {
122              if (isCalibrated[i]) continue; // Skip if already done
123              tcaselect(i);
124              delay(500);
125              while (sensoren.getFrame(tempFrame) != 0) {
126                  delay(50);
127              }
128              for (int p = 0; p < 768; p++) {
129                  backgroundFrames[i][p] = tempFrame[p];
130              }
131              saveBackground();
132              isCalibrated[i] = true;
133          }
134      }
```

```
135  void messageSuccess() {
136      Serial.println("ok");
137  }
138
139
140  void messageFail() {
141      Serial.println("fejl");
142  }
143
144
145
146  };
```

Figur 61 - viser calibrate() metoden samt de 2 metoder der anvender til at informerer om sucess og fejl til GUI.

Figur 61 viser metoden calibrate(). Denne metode forklares i detalje i Bilag 1.C - Design.

9.2.6 PMSControl

Figur 62 viser starten af klassen PMScontrol. Klassen PMScontrol har til formål at håndtere hovedlogikken for IR-sensorer og deres associerede ”rum” (som er repræsenteret af 3 LED sæt, 1 for hvert simuleret rum).

Denne klasse er forklaret i detalje i Bilag 1.C – Design.

```
8   class PMScontrol {
9       private:
10          bool relayState[3] = {true, true, true};
11
12      public:
13          void process(IRsensor &sensoren, InitierPMS &config, PowerSensor &powerSensor) {
14              if (!config.systemReady) return;
15
16              double currentCapacity = powerSensor.getCapacityPct();
17              float mlxFrame[768];
18
19              for (uint8_t i = 0; i < 3; i++) {
20                  if (!config.isCalibrated[i]) continue;
21
22                  config.tcaselect(i);
23                  if (sensoren.getFrame(mlxFrame) != 0) continue;
24
25                  bool targetRelayState = config.sensor[i].getActivity(mlxFrame, config.initOn_);
26
27                  if (currentCapacity < profileData.getThreshold(i)) {
28                      targetRelayState = false;
29                  }
30
31                  bool currentRelayState = relayState[i];
32
33                  if (targetRelayState != currentRelayState) {
34                      digitalWrite(config.RELAY_PINS[i], targetRelayState ? LOW : HIGH);
35
36                      relayState[i] = targetRelayState;
37                  }
38              }
39              delay(50);
40          }
41  };
```

Figur 62 - viser de private og public attributter samt metoder for PMScontrol klasse.

9.2.7 SetPMS

Mark

Klassen SetPMS bruges til at opdatere threshold i ProfileData. Oversigt over klassen kan ses på figur 63.

```
class SetPMS {
public:
void updateThresholds();
};
```

Figur 63 - SetPMS klasse oversigt

SetPMS metoder:
void updateThresholds():

Metoden oversætter en string på form ”Profile X|x,y,z\n” til threshold-værdier.

Først tjekker metoden om der er modtaget noget fra Raspberry Pi. Hvis der ikke er modtaget noget, så stoppes metoden.

```
void SetPMS::updateThresholds() { // Tager strings på form "Profile 1|0,0,0\n"
    if (!Serial.available()) return;
```

Figur 64 - Udsnit af source kode til updateThresholds()

Hvis der er modtaget noget fra Raspberry Pi, så gemmes metoden i en string *line*. Herefter tjekkes om denne string starter med “Profile”, da dette er hvad der forventes hvis stringen skal bruges til at opdatere thresholds. I prototypen modtages kun én type besked fra Raspberry Pi, så det er ikke nødvendigt at tjekke om der modtages string på rigtig form. Men det er god praksis og giver mulighed for at udvide med mere kommunikation fra Raspberry Pi til ESP32 i fremtiden.

```
String line = Serial.readStringUntil('\n');
if (!line.startsWith("Profile ")) return;
```

Figur 65 - Udsnit af source kode til updateThresholds()

Metoden findes index af ”|”, hvis det ikke indgår i modtaget string, så stoppes oversætning.

```
int separator = line.indexOf('|');
if (separator == -1) return; // separator bliver -1, hvis der ikke er nogen | i strengen.
```

Figur 66 - Udsnit af source kode til updateThresholds()

Der tjekkes om profil nummeret er mellem 1 og 5, som er de 5 profiler mulige profiler der er.

```
int profileNum = line.substring(8, separator).toInt();
if (profileNum < 1 || profileNum > 5) return; // Validér profilnummer
```

Figur 67 - Udsnit af source kode til updateThresholds()

Der tjekkes nu om den modtaget besked passer overens med det der forventes at modtage. Nu skal der hives nogen tal ud af beskeden.

Der modtages en besked ”Profile X|x,y,z”. Først fjernes Profile X|, så kun x,y,z er tilbage.

```
String values = line.substring(separator + 1);
```

Figur 68 - Udsnit af source kode til updateThresholds()

Der findes index af komma et og to.

```
int firstComma  = values.indexOf(',');
int secondComma = values.indexOf(',', firstComma + 1);
```

Figur 69 - Udsnit af source kode til updateThresholds()

Så hives tal før første komma ud, dernæst tal mellem de 2 kommaer og til sidst tal efter andet komma.

Disse udhævede tal bruges som parameter til ProfileData metodes editProfile().

```
    String val0 = values.substring(0, firstComma);
    String val1 = values.substring(firstComma + 1, secondComma);
    String val2 = values.substring(secondComma + 1);

    profileData.editProfile(0, val0.toFloat());
    profileData.editProfile(1, val1.toFloat());
    profileData.editProfile(2, val2.toFloat());
}
```

Figur 70 - Udsnit af source kode til updateThresholds()

9.2.8 TerminatePMS

TerminatePMS klassen har til formål at sikre, at PMS lukkes ned, og at strømmen til rum ikke længere bliver styret og overvåget af PMS. I InitPMS klassen er der defineret 2 attributter, som angiver, hvorvidt systemet er initieret, og hvorvidt systemet er klar til brug. Disse 2 attributter sættes til true ved succesfuld initiering og systemstart, og sættes herefter til false, når systemet slukker. Slukningen af systemet initieres, når ADC_pin2 registrerer, at der ikke længere flyder spænding fra Emergency Power som kan ses i **Fejl! Henvisningskilde ikke fundet.**.

```
47      if (!powerMonitor.emergencyPowerNeeded()) {
48          terminator.powerDown(initiator);
49      }
```

Figur 71 - viser hvorledes TerminatePMS anvendes i main.

```
1   #pragma once
2   #include "InitPMS.h"
3   class TerminatePMS {
4   public:
5   void powerDown(InitPMS &config);
6   };
```

Figur 72 - viser .h filen for TerminatePMS.

Fejl! Henvisningskilde ikke fundet. viser klassens ene metode som laves public, da den skal k unne anvendes af andre klasser (*public: void powerDown(InitPMS &config);*. Der inkluderes også en reference til et objekt af InitPMS klassen kaldet config.

```
1   #include "InitPMS.h"
2   #include "TerminatePMS.h"
3   #include "PowerSensor.h"
4
5   void TerminatePMS::powerDown(InitPMS &config){
6     config.setInitOn(false);
7     config.systemReady = false;
8     for (int i = 0; i < 3; i++) {
9         digitalWrite(config.RELAY_PINS[i], LOW);
10    }
11    powerMonitor.resetCapacity();
12    Serial.println((int)100);
13    Serial.println("idle");
14
15  }
```

Figur 73 - viser TerminatePMS klassens .cpp fil.

Fejl! Henvisningskilde ikke fundet. viser .cpp filen for klassen TerminatePMS. Der er en m etode, som returnerer void ((void TerminatePMS::powerDown(InitPMS &config)). I denne

metode sættes 2 attributer fra InitPMS til at være ”false”. Herefter slukkes al strøm til rummene (lysene slukkes), og der printes til sidst til Raspberry Pi beskeden ”idle”. Denne besked trigger pop up beskeden ”Power has returned to the building. PMS is turned off”. Dette skal repræsentere, at blackoutet, der skaber strømsvigtet, er overstået, og at en stabil strømkilde igen er tilgængelig.

TerminatePMS aktiveres, når betingelsen (!Powermonitor.isEmergencyPowerNeed()) er sand, som vist i **Fejl! Henvisningskilde ikke fundet.**. Denne betingelse refererer til funktionen isEmergencyPowerNeed(), som tjekker om, der kommer strøm fra den simulerede Emergency Power. Hvis der ikke detekteres strøm, er Emergency Power enten løbet tør for brændstof/batteri, eller den er blevet slukket når strøm returnerer til systemet.

Uanset hvilken situation det er, skal systemets aktive overvågning af bygningens rum sluttes, og PMS skal returneres til passiv overvågning af Emergency Power output i tilfælde af, at det skal aktiveres igen. Kapaciteten af Emergency Power sættes også her tilbage til sin startkapacitet, så en simulering kan køres på ny.

10 Implementering og modultest

Modultest kan findes i Bilag 1.D - Test.

11 Integrationstest

Den største udfordring under integration fasen var, at kommunikationen mellem ESP32 og Raspberry Pi ikke fungerede. Alt logik kørte uden fejl på ESP32 separat og GUI kørte uden problemer, men fik ikke opdateret kapaciteten på home pagen. Efter mange timer og mange forskellige løsningsforslag fra diverse GAI blev fejlen fundet. Fejlen lå i, at USB-porten blev initialiseret, mens serielbiblioteket blev importeret. Dette blev løst med en connect funktion, som blev kaldt senere i opsætningen. Herefter kørte kommunikationen fra ESP32 til Raspberry Pi uden fejl, men det at sende thresholds virkede stadig ikke.

Forklaring på dette var at funktionen til at sende seriel besked sluttede på ”\n”, og beskeden, der blev skrevet til denne funktion, sluttede også på ”\n”. Dette resulterede i, at beskeden havde to gange ”\n”. Da denne blev fjernet fra beskeden, virkede det fejlfrit.

Udover kommunikationen mellem GUI’en og ESP32’en blev thresholds også integreret ind i den eksisterende logik, som blev testet i modultest for softwarekomponenten af IR sensoren. I modultest blev der kun testet, at kameraerne kunne detektere individer og aktivere kameraerne, når der blev detekteret individer. I integrationen af power sensoren og IR sensoren blev thresholds implementeret og aktivering af lys og relæene blev betinget af kapaciteten af den associerede Emergency Power.

12 Accepttest

12.1 Accepttestspecifikation for funktionelle krav

Accepttest specifikationen for funktionelle krav kan findes i Bilag 1.D - Test. Se eventuelt afsnittet "Resultater" for en oversigt over godkendte/ikke godkendte krav.

12.2 Accepttestspecifikation for ikke-funktionelle krav

Accepttest specifikationen for ikke-funktionelle krav kan findes i Bilag 1.D - Test. Se eventuelt afsnittet "Resultater" for en oversigt over godkendte/ikke godkendte krav.

13 Resultater

13.1 Resultater for accepttest for use cases

Use case	Punkter godkendt
1.	6/6
2.	2/2
3.	2/2
4.	5/5
5.	2/2

Tabel 16 - Antal godkendte punkt i hver use case

13.2 Resultater for accepttest for ikke-funktionelle krav

Ikke funktionelle krav	Punkter godkendt
U1	Ikke godkendt
U2	Godkendt
U3	Godkendt
U4	Ikke godkendt
U5	Godkendt
U6	Godkendt
U7	Godkendt
R1	Godkendt
R2	Godkendt
R3	Godkendt
R4	Ikke godkendt
R5	Ikke godkendt
P1	Godkendt
P2	Godkendt
P3	Godkendt
P4	Godkendt
S1	Godkendt

Tabel 17 - Godkendte punkter fra ikke funktionelle krav

14 Diskussion af resultater

Generelt er projektet kommet i mål med kernefunktionaliteten af produktet, idet alle funktionelle krav er blevet opfyldt. Kun 4 ud af 17 punkter af de ikke-funktionelle krav blev ikke godkendt.

Årsagen til at ikke alle punkter for den ikke-funktionelle accepttest var godkendt, skyldes det højere fokus på at få produktet til at virke som specificeret i de funktionelle krav, frem for et fokus på at opfylde alle ikke-funktionelle krav.

Der har været fokus på at fremstille en prototype for et produkt, der i fremtiden kan implementeres i den virkelige verden.

Dog var alle de punkter, der ikke blev nået, U4, R1 og R5, allesammen punkter der ikke var "must" og blev derfor nedprioriteret.

Helt generelt kan der ses ud fra projektet, at der var høje ambitioner og ønsker om at lave et produkt, som holder sig så tæt til virkeligheden som muligt. Men en mangel på erfaring med hensyn til at formulere use cases, ikke-funktionelle krav samt accepttests resulterede i, at det ikke lykkedes at dække alle relevante krav samt at teste disse krav i accepttesten. Men med erfaringen der blev opnået igennem projektet, vil fremtidige projekter være veldefinerede samt realistiske med hensyn til projektgruppens ønsker og ambitioner.

15 Perspektivering / Fremtidigt arbejde

Projekt Blackout er en prototype af et system, der i den virkelige verden ville kunne implementeres hos kritiske bygninger, og derfor har vi ikke i vores projekt medtaget alle de funktioner, som vi ville have med i systemet på stor skala.

Af disse kan f.eks. nævnes implementering af en sikkerhedsmekanisme, som sikrer automatisk genstart af I^2C kommunikation. Hvis der mellem mikrocontrolleren og sensorerne via multiplexeren skulle opstå problemer med dette, ville der blive sendt en specifik besked om det til display på GUI.

Derudover har mange industrielle applikationer flere måder at tilgå den samme funktion på, hvor vi ligeledes forestiller os, at en lignende funktion skulle inkorporeres i Projekt Blackout som færdigt system. Her har vi særligt haft en "switch" med i tankerne, enten som en fysisk kontakt i ethvert rum i den kritiske bygning eller en knap, som kan tilgås via GUI'en. Formålet med denne ville være, at man eventuelt kunne overskrive vores system og dermed at kunne slukke for strømmen i et, ellers prioriteret, rum, hvis prioritering af det valgte rum synes unødvendigt. Dette kan være foretrukket, hvis det af User vurderes, at der er højere prioritet at have længere tid med strøm i et andet rum. Fordelen ved dette er, at der leveres et mere brugertilpasset system, og samtidig gør en "switch" systemet mere brugervenligt.

På trods af dette valgte vi ikke at implementere dette grundet tidsnød, hvilket betød, at vi prioriterede at implementere andre, mere kritiske funktioner i vores system, men det er noget, vi har overvejet at indføre i vores system på stor skala. Samtidig diskuterede vi også, hvorvidt implementeringen af denne "switch" er nødvendig i nedskaleringen af systemet, idet vores scenarie blot består af 3 rum, hvor man nemmere blot kan ændre threshold for et givent rum til 100%. Da vi, som førnævnt, var under tidspres, vurderede vi, at denne funktion ikke var af høj prioritet.

Udover en "switch" med dette formål, ville vi inkorporere en "switch", der muliggør manuel sluk og tænd af hele PMS. Vi anser en sådan funktion som vigtig for systemet, idet det gør det muligt for User at genstarte systemet i tilfælde af uhensigtsmæssig adfærd. Dette er desuden brugt i stor grad i andre store systemer, hvor hardware og software arbejder tæt sammen.

Der var også i løbet af processen gjort nogle tanker om en implementering af en brugerdefineret tid, der skulle bestemme, hvor lang tid der skal gå før, strømmen slukker i et rum. Denne brugerdefinerede tid var tænkt til at skulle styres fra en slider i GUI "Settings" menuen. Implementeringen blev dog droppet pga. mangel på tid.

Helt fra start var der også tanker gjort om, hvad man kunne have som ekstra mål, hvis der var tid til overs. Et af de mål var en implementering af en statistikunderside, som altså kunne indeholde samlet og behandlet data fra sensorerne. Ideen var, at statistikundersiden kunne hjælpe med at "guide" User til at optimere systemet til at spare mest strøm muligt, ved at vise hvilke rum ikke blev brugt ofte og bruge den information til at foreslå ændringer i den aktive profil.

Prototypen anvender ikke strømsensoren til at tjekke, om Emergency Power er aktivt, og dermed om systemet bør være aktivt. Til et fremtidsprodukt, der skulle ud i den virkelige verden, kan den anvendes til denne funktion, især hvis det er en travl offentlig bygning som f.eks. et hospital. Da der i travle bygninger aldrig vil være en situation, hvor der ikke er aktivitet i nogen af rummene, vil der altid løbe en strøm, som strømsensoren kan opfange. Dermed ved man, at når der ikke længere løber strøm fra Emergency Power, må det betyde, at det almindelige elnet er returneret.

Hvis prototypen skulle ud i industrien, skal man også tænke over, hvordan en reel nødstrømsløsning ser ud. Her er en typisk løsning en UPS, som slår til i det øjeblik, strømsvigt rammer. En UPS-løsning kan bestå af batterier, for eksempel lithium-ion-batterier[20], som kan holde bygningen kørende kortvarigt. Herefter kan en eventuel generator tage over[21], f.eks. en dieselgenerator, som kan holde strømmen kørende i en længere periode. PMS kunne bruges på sådan en sammensætning ved at regne på den samlede kapacitet i mAh. Det kan simpelt gøres

[20]Ukendt forfatter, "Eaton Lithium-ion Batteries"
[21]Altini, Tom," Critical Power Infrastructure: Generator and UPS"

ved at kigge på UPS'ens specifikationer, og for en dieselgenerator kan man regne på, hvor meget den producerer per kilo diesel, og regne ud fra hvor meget diesel, der er lagret til den. På den måde vil PMS forholdsvis simpelt kunne overføres til den virkelige verden.

16 Konklusion

I løbet af dette projekt, blev der udarbejdet en prototype, hvis funktionalitet fungerer og løser problemet om, hvordan det sikres at begrænset energi udnyttes optimalt så essentielle funktioner holdes kørende længst muligt. Derudover opfylder prototypen de funktionelle krav, der blev sat i starten af projektet. Altså kan prototypen detektere, regulere, og ud fra aktiviteten, samt ud fra rummets threshold, regulere tilførslen af strøm. På denne måde bliver der ikke brugt unødigt strøm i rum, hvor der ingen aktivitet er. Derudover når strømniveauet falder, frakobler strømtilførslen til rum som, ud fra Users egen vurdering, ikke er essentielle.

17 Bilag

1. Projekt
   1.A - Analyse
   1.B – Arkitektur
   1.C – Design
   1.D – Test
   1.E – PowerSensor ”Data taget med ESP32 til Power Sensor kalibration”
   1.F – Datablade
   1.G - Testmodul kode
   1.H - Code
2. Proces
   2.A – Procesbeskrivelse
   2.B – Samarbejdskontrakt
   2.C – Timeline
   2.D - Møder
   2.E - Projekt idéer
   2.F - TODO

18 Kildeliste

Forfatter: Altini, Tom
Title: “Critical Power Infrastructure: Generator and UPS”
Besøgt: 20-05-2026
udgivet: 09-12-2025
Link: https://ravenvolt.com/blog/critical-power-infrastructure-generator-and-ups/

Forfatter: Ukendt
Title: “Eaton Lithium-ion Batteries”
Besøgt: 20-05-2026
Udgivet: Ikke angivet
Link: https://www.eaton.com/dk/da-dk/products/backup-power-ups-surge-it-power-distribution/3-phase-ups-accessories/eaton-lithium-ion-batteries-emea.html

Forfatter: Craig, Arthur D.
Title: “Temperature Sensation”
Besøgt: 18-05-2026

Udgivet: 2011
Link: https://www.sciencedirect.com/science/chapter/referencework/pii/B9780080450469019227

Forfatter: ATO.com
Title: ”Difference Between Relays and Transistors”
Besøgt: 04-05-2026
Udgivet: 16-04
Link: https://www.ato.com/difference-between-relays-and-transistors

Forfatter: Ukendt
Title: ”Galvanisk Adskillelse: En dybdegående guide til sikker isolation i teknologi og transport”
Besøgt: 04-05-2026
Udgivet: 05-01-2026
Link: https://a-website.dk/galvanisk-adskillelse/

Forfatter: Stock, Michael
Title: “Galvanic Isolation: Purpose and Methodologies”
Besøgt: 20-05-2026
Udgivet: 12-01-2016
Link: https://www.allaboutcircuits.com/technical-articles/galvanic-isolation-purpose-and-methodologies/

Forfatter: Ouiyzme, Larbi
Title: ”Understanding I2C Protocol, Addressing, and How to Build an I2C Scanner for ESP Boards”
Besøgt: 20-05-2026
Udgivet: 06-10-2026
Link: https://larbi-ouiyzme.medium.com/understanding-i2c-protocol-addressing-and-how-to-build-an-i2c-scanner-for-esp-boards-3f4d93af3b24

Forfatter: Rembor, Katni
Title: “Adafruit MLX90640 IR Thermal Camera”
Besøgt: 10-04-2026
Udgivet: 22-10-2025
Link: https://learn.adafruit.com/adafruit-mlx90640-ir-thermal-camera

Forfatter: Söderby, Karl
Title: ”Using the Serial Monitor tool”
Besøgt: 26-03-2026
Udgivet: 12-02-2024

Link: https://docs.arduino.cc/software/ide-v2/tutorials/ide-v2-serial-monitor/

Eksterne biblioteker anvendt til IR-sensor klassen:

Eksternt bibliotek: ”Adafruit_MLX90640”
Link: https://github.com/adafruit/Adafruit_MLX90640.git

Eksterne biblioteker anvendt til GUI klassen:

Eksternt bibliotek: ”tkinter”

Link: https://github.com/python/cpython/tree/3.14/Lib/tkinter

Eksternt bibliotek: ”tkinter.ttk”

Link: https://github.com/python/cpython/blob/3.14/Lib/tkinter/ttk.py

Eksternt bibliotek: ”threading”

Link: https://github.com/python/cpython/blob/3.14/Lib/threading.py

Eksternt bibliotek: ”serial”

Link: https://github.com/pyserial/pyserial/